

ⓘ This is a copy of a chat between Claude and Abuzar. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. Report

is multi level algorithm the algorithm are RLE,LZW,Huffman File compressor a good idea for Java OOP project semester end is this good length project and unique practical in which i have to think and understand everything rather than using api or libraries to do my [work. Is](<http://work.is>) this better and bigger than traditional mangement system projects.

Multi-Level Compression Algorithm Project — Assessment

Yes, this is an excellent project choice. Here's a full breakdown:

Why It's Better Than Traditional Management Systems

Aspect	Management System (CRUD)	Your Compression Project
Thinking required	Low — standard patterns	High — algorithm logic
OOP depth	Shallow	Deep — real inheritance, polymorphism
Uniqueness	Very common	Rare among students
Practical value	Generic	Real-world CS concept
Coding challenge	Minimal	Significant
Impressiveness	Low	High

OOP Concepts You'll Naturally Use

- **Abstraction** — `CompressionAlgorithm` abstract base class
- **Inheritance** — `RLE`, `LZW`, `Huffman` extend it
- **Polymorphism** — call `compress()` on any algorithm via the same interface
- **Encapsulation** — each algorithm hides its internal logic

- **Design Patterns** — Strategy pattern fits perfectly here
-

Project Complexity & Length — Is It Right?

Yes — it's the perfect semester-end size. Not too small, not overwhelming.

- **RLE** — Easiest, good warm-up (2-3 days)
- **LZW** — Medium, dictionary-based thinking (4-5 days)
- **Huffman** — Hardest, requires trees + priority queues (1 week)
- **Multi-level pipeline** — Chaining them together (2-3 days)
- **File I/O + UI** — Reading/writing binary files, simple CLI or GUI (2-3 days)

Total: ~3-4 weeks of solid work, which is ideal.

What Makes It Genuinely Practical

- You'll understand **why ZIP, PNG, and GIF** work the way they do
 - You'll build a **real binary file compressor** that actually reduces file size
 - You'll implement **Huffman trees from scratch** — a classic CS interview topic
 - You'll think about **bit manipulation**, not just object creation
-

Recommended Architecture

```
CompressionAlgorithm (abstract)
|
├─ RLECompressor
├─ LZWCompressor
├─ HuffmanCompressor
|
MultiLevelCompressor (uses Strategy pattern)
|
FileCompressorApp (main entry — handles File I/O)
```

Honest Caveats

- **Huffman encoding** requires understanding binary trees and priority queues — give it real time

- **LZW** has dictionary-building logic that needs careful thinking
- **Bit-level I/O** in Java (writing actual compressed bits to a file) is tricky — plan for it

Bottom line: This project demonstrates real computer science understanding, strong OOP design, and practical knowledge. It will stand out significantly compared to a library/inventory/student management system. Go for it.

like is this project big enough to cover my 3 weeks or small that can be made quickly. Because im thinking will sir say that this is short project it could be made very quickly

Honest Size Assessment

Your concern is valid — let me be real with you.

The Truth About Difficulty

If you implement it **superficially**, yes — a basic version can be done in 3–5 days. That's the risk. Your sir might think:

"RLE is just count-repeat, Huffman is textbook code, this is copy-paste work."

BUT — if you build it the **RIGHT** way, it becomes a genuinely large project. The difference is in the depth.

What Makes It SHORT vs WHAT Makes It BIG

Short Version 

Big Version 

RLE on text only

RLE on any binary file

Copy Huffman tree code

Build tree, encoder, AND decoder from scratch

Just compress strings

Actually write/read compressed `.bin` files

One algorithm at a time

Pipeline: file → RLE → LZW → Huffman → output

Short Version 

Big Version 

No comparison	Show compression ratio, speed, file size stats
Console print only	Proper GUI (JavaFX or Swing) with file chooser
No error handling	Handle corrupt files, edge cases
Single file	Compress entire folders

My Honest Recommendation — Add These to Make It Undeniably BIG

1. Full Compression + Decompression (mandatory)

Not just compress — fully decompress back to original. This doubles the work and proves correctness.

2. Performance Comparison Dashboard

After compressing, show a report:

Algorithm	Original	Compressed	Ratio	Time Taken
RLE	500 KB	320 KB	36%	12ms
Huffman	500 KB	180 KB	64%	45ms
RLE + Huffman	500 KB	160 KB	68%	57ms

3. JavaFX GUI with File Browser

Drag and drop files, choose algorithm, see live progress bar — this alone adds a full week of work.

4. Compress ANY File Type

Images, PDFs, text — not just `.txt` files. Binary file handling is genuinely hard.

5. Custom File Format

Save compressed output as `.mlc` (Multi Level Compressed) with a custom header storing metadata — this shows systems thinking.

Realistic Timeline (Done Properly)

Week 1 — Core Algorithms

Days 1-2 → RLE (compress + decompress + tests)

Days 3-4 → LZW (dictionary logic, compress + decompress)

Days 5-7 → Huffman (tree, priority queue, bit encoding, decoding)

Week 2 — Integration & File Handling

Days 1-2 → Binary file I/O (actual bit-level reading/writing)

Days 3-4 → Multi-level pipeline (chain algorithms)

Days 5-7 → Custom .mlc format + compression stats engine

Week 3 — Polish & GUI

Days 1-3 → JavaFX GUI (file chooser, progress, results table)

Days 4-5 → Folder compression support

Days 6-7 → Testing, fixing, documentation + demo prep

That is a genuinely full 3 weeks.

What To Tell Your Sir

Frame it as:

"I am implementing three compression algorithms from scratch at the binary level, with full compression and decompression, a multi-level pipeline, performance benchmarking, and a GUI — without using any compression libraries."

That framing makes it **impossible** to call it a small project.

Bottom Line

The project size lives or dies by your depth, not your topic. Done properly with GUI + binary I/O + decompress + benchmarking — this is bigger and harder than 90% of management systems. Done lazily, it's a weekend project. The choice is yours.

Can you provide me full road map for Big version

Full Roadmap — Big Version Multi-Level File Compressor

Project Structure First

```

MultiLevelCompressor/
|
├─ src/
|   ├─ algorithms/
|   |   ├─ CompressionAlgorithm.java      (abstract base)
|   |   ├─ RLECompressor.java
|   |   ├─ LZWCompressor.java
|   |   └─ HuffmanCompressor.java
|   |
|   ├─ core/
|   |   ├─ MultiLevelCompressor.java      (pipeline)
|   |   ├─ CompressionResult.java        (stats model)
|   |   ├─ FileHeader.java                (custom .mlc format)
|   |   └─ CompressionPipeline.java       (strategy pattern)
|   |
|   ├─ io/
|   |   ├─ FileReader.java
|   |   ├─ FileWriter.java
|   |   └─ BitStream.java                 (bit-level I/O)
|   |
|   └─ gui/
|       ├─ MainWindow.java
|       ├─ CompressionPanel.java
|       ├─ ResultsDashboard.java
|       └─ ProgressPanel.java
|
|   └─ utils/
|       ├─ CompressionStats.java
|       ├─ BenchmarkEngine.java
|       └─ Logger.java
|
├─ test/
|   ├─ RLETest.java
|   ├─ LZWTest.java
|   └─ HuffmanTest.java
|
└─ output/                                (compressed .mlc files)

```

WEEK 1 — Core Algorithms From Scratch

Day 1-2 — Abstract Base + RLE

Goal: Build the foundation and first algorithm

Step 1 — Abstract Base Class

```
java

// CompressionAlgorithm.java
public abstract class CompressionAlgorithm {
    protected String algorithmName;
    protected long compressionTime;

    public abstract byte[] compress(byte[] data);
    public abstract byte[] decompress(byte[] data);
    public abstract String getAlgorithmName();

    // Common utility – calculate ratio
    public double getCompressionRatio(byte[] original, byte[] compressed) {
        return (1.0 - ((double) compressed.length / original.length)) * 100;
    }
}
```

Step 2 — RLE Algorithm (Binary Level)

Concept:

Input bytes: [A, A, A, A, B, B, C, A, A]

Encoded: [4, A, 2, B, 1, C, 2, A]

Store: count (1 byte) + value (1 byte) pairs

Handle count overflow (max 255 per run)

Handle ALL file types – not just text

What to implement:

- `compress(byte[] data)` — scan runs, write count+value pairs
- `decompress(byte[] data)` — read pairs, expand back
- Edge cases: single bytes, runs of 255+, empty files
- Test on text files AND image files

Checklist Day 1-2:

- ☐ Abstract class done
- ☐ RLE compress works on text
- ☐ RLE compress works on binary/image files
- ☐ RLE decompress restores file perfectly

- ☐ Unit test: compress → decompress → compare original

Day 3-4 — LZW Algorithm

Goal: Dictionary-based compression from scratch

Concept to understand:

LZW builds a dictionary as it reads:

Start: dictionary has all 256 single bytes

Read "AB" → not in dict → add "AB"=256, output code for "A"

Read "BC" → not in dict → add "BC"=257, output code for "B"

Read "ABC" → not in dict → add "ABC"=258, output code for "AB"=256

Dictionary grows automatically – no need to store it in file!

Decoder rebuilds the same dictionary during decompression.

What to implement:

- `compress(byte[] data)` — build dictionary, output integer codes
- `decompress(byte[] data)` — rebuild dictionary, decode codes back
- Store codes as 12-bit or 16-bit integers
- Handle dictionary reset when it gets full (4096 or 65536 entries)
- Convert code stream back to byte array properly

Checklist Day 3-4:

- ☐ Dictionary initialization (256 entries)
- ☐ Encoding loop with dictionary building
- ☐ Code output as byte stream
- ☐ Decoder with dictionary reconstruction
- ☐ Test: LZW on text, then on repeated-pattern binary files

Day 5-7 — Huffman Coding (Hardest Part)

Goal: Frequency-based binary tree encoding

Concept breakdown:

Step 1 – Frequency Count:

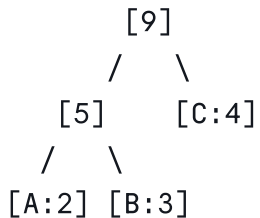
"AABBBCCCC" → A:2, B:3, C:4

Step 2 – Build Min Priority Queue:

[A:2] [B:3] [C:4]

Step 3 – Build Huffman Tree:

Combine two smallest repeatedly:



Step 4 – Assign Codes:

C → 1 (1 bit)

A → 00 (2 bits)

B → 01 (2 bits)

Step 5 – Encode:

"AABBBCCCC" → 00 00 01 01 01 1 1 1 1

Step 6 – Save tree in file header (for decoding)

Step 7 – Decode using tree traversal

Classes needed inside Huffman:

```
java

// Inside HuffmanCompressor.java
private class HuffmanNode implements Comparable<HuffmanNode> {
    byte value;
    int frequency;
    HuffmanNode left, right;
    // compareTo by frequency for PriorityQueue
}
```

What to implement:

- Frequency table builder
- Min-heap priority queue (use Java's `PriorityQueue`)
- Tree builder (combine nodes until one root)
- Code table generator (tree traversal → bit strings)
- Encoder (replace each byte with its bit code)
- Bit packing (pack bit strings into actual bytes)

- Tree serializer (save tree structure into file for decoding)
- Decoder (read tree, traverse with bits to recover bytes)

Checklist Day 5-7:

- ☐ Frequency counter works
 - ☐ HuffmanNode with Comparable
 - ☐ Tree builds correctly
 - ☐ Code table generated
 - ☐ Bits packed into bytes correctly
 - ☐ Tree saved to and loaded from bytes
 - ☐ Full decompress restores original
 - ☐ Test on text, test on image
-

WEEK 2 — Integration, File I/O & Pipeline

Day 1-2 — Binary File I/O + BitStream

Goal: Read/write ANY file at bit level

BitStream class — most important utility:

```
java

public class BitStream {
    // Writing bits one by one, packing into bytes
    public void writeBit(int bit) { ... }
    public void writeBits(String bitString) { ... }
    public byte[] toByteArray() { ... }

    // Reading bits from byte array
    public int readBit() { ... }
    public String readBits(int count) { ... }
}
```

FileHandler:

```
java

public class FileHandler {
    public byte[] readFile(String path)           // read any file as bytes
    public void writeFile(String path, byte[])    // write bytes to any file
    public String getFileExtension(String path)
```

```
public long getFileSize(String path)
}
```

Checklist:

- ☐ Read any file (text, image, PDF) as byte array
- ☐ BitStream writes and reads bits correctly
- ☐ Padding bits handled (last byte may not be full)
- ☐ Write byte array back to file perfectly

Day 3-4 — Custom .mlc File Format

Goal: Design your own compressed file format

File Header Structure:

.mlc File Layout:

MAGIC NUMBER	(4 bytes)	"MLC\0" – identifies file type
VERSION	(1 byte)	format version
ALGORITHM IDs	(3 bytes)	which algorithms used + order
ORIGINAL SIZE	(8 bytes)	original file size (long)
ORIGINAL EXT	(4 bytes)	.txt, .png, .pdf etc
TIMESTAMP	(8 bytes)	when compressed
CHECKSUM	(4 bytes)	CRC32 for integrity check
HUFFMAN TREE	(variable)	if Huffman used, tree bytes
COMPRESSED DATA		

java

```
public class FileHeader {
    public static final byte[] MAGIC = {'M', 'L', 'C', 0};
    private byte version;
    private byte[] algorithmIds;
    private long originalSize;
    private String originalExtension;
    private long timestamp;
    private int checksum;

    public byte[] serialize() // header → bytes
    public static FileHeader deserialize(byte[]) // bytes → header
}
```

Checklist:

- ☐ Magic number written and verified on read
 - ☐ All metadata stored correctly
 - ☐ Huffman tree embedded in header
 - ☐ CRC32 checksum computed and verified
 - ☐ Corrupt file detected and reported
-

Day 5-7 — Multi-Level Pipeline

Goal: Chain algorithms together, Strategy pattern

Pipeline Design:

java

```
public class CompressionPipeline {
    private List<CompressionAlgorithm> stages;

    public void addStage(CompressionAlgorithm algorithm) {
        stages.add(algorithm);
    }

    public byte[] compress(byte[] data) {
        byte[] current = data;
        for (CompressionAlgorithm algo : stages) {
            current = algo.compress(current);
            // record stats at each stage
        }
        return current;
    }

    public byte[] decompress(byte[] data) {
        byte[] current = data;
        // decompress in REVERSE order
        for (int i = stages.size()-1; i >= 0; i--) {
            current = stages.get(i).decompress(current);
        }
        return current;
    }
}
```

Preset Pipelines:

Pipeline A: RLE only
Pipeline B: Huffman only
Pipeline C: LZW only
Pipeline D: RLE → Huffman (good for text)
Pipeline E: LZW → Huffman (good for mixed files)
Pipeline F: RLE → LZW → Huffman (maximum compression)

BenchmarkEngine:

```
java

public class BenchmarkEngine {
    public CompressionResult benchmark(
        byte[] originalData,
        CompressionPipeline pipeline
    ) {
        long start = System.nanoTime();
        byte[] compressed = pipeline.compress(originalData);
        long compressTime = System.nanoTime() - start;

        start = System.nanoTime();
        byte[] decompressed = pipeline.decompress(compressed);
        long decompressTime = System.nanoTime() - start;

        return new CompressionResult(
            originalData.length,
            compressed.length,
            compressTime,
            decompressTime,
            Arrays.equals(originalData, decompressed) // integrity check
        );
    }
}
```

Checklist:

- ☐ Pipeline chains algorithms correctly
- ☐ Reverse decompression works
- ☐ All 6 preset pipelines work
- ☐ BenchmarkEngine records time + ratio
- ☐ Integrity check confirms perfect restoration

Day 1-3 — JavaFX GUI

Goal: Full graphical interface

4 Screens to Build:

Screen 1 — Main Window

```
|   Multi-Level File Compressor |
|                                     |
|  [ Drag & Drop File Here ]         |
|  or [ Browse File ] [ Browse Folder ] |
|                                     |
|  Selected: document.pdf (2.4 MB)    |
|                                     |
|  Algorithm Pipeline:                |
|  ○ RLE Only                        |
|  ○ Huffman Only                    |
|  ○ LZW Only                        |
|  ● RLE → Huffman                   |
|  ○ LZW → Huffman                   |
|  ○ RLE → LZW → Huffman (Max)       |
|                                     |
|  [ COMPRESS ]      [ DECOMPRESS ]  |
```

Screen 2 — Progress Panel

```
|  Compressing document.pdf...        |
|                                     |
|  Stage 1: RLE      [=====] Done  |
|  Stage 2: Huffman  [====] 40%      |
|                                     |
|  Processing 1.2 MB / 2.4 MB         |
```

Screen 3 — Results Dashboard

```
|  Compression Complete ✓             |
|                                     |
|  Original Size   : 2,400 KB         |
```

```
| Compressed Size:      890 KB |
| Space Saved   :  1,510 KB  (62.9%) |
| Time Taken    :      234 ms |
| Integrity Check:  PASSED ✓ |
|
| Pipeline Breakdown: |
| After RLE     → 1,800 KB  (25% saved) |
| After Huffman →   890 KB  (50% more) |
|
| [ Save File ] [ Compress Another ] |
```

Screen 4 — Benchmark Comparison

```
| Algorithm Comparison for test.txt |
|                                     |
| Algorithm      Ratio   Time   Size |
| -----|-----|-----|-----|
| RLE            25%     8ms   1800KB |
| LZW            55%    23ms   1080KB |
| Huffman        63%    41ms    888KB |
| RLE+Huffman    66%    49ms    816KB |
| LZW+Huffman    71%    67ms    696KB |
| RLE+LZW+Huff   73%    89ms    648KB |
|                                     |
| [Bar Chart Visual] |
```

Day 4-5 — Folder Compression

Goal: Compress entire folders, not just single files

```
java

public class FolderCompressor {
    // Flatten folder into one byte stream:
    // [filename_length][filename][filesize][filedata] repeated

    public byte[] packFolder(File folder)
    public void unpackFolder(byte[] data, String outputPath)
}
```

Folder Structure Stored:

```
|— report.txt      → stored with name + size + data
|— image.png      → stored with name + size + data
|— notes.txt      → stored with name + size + data
```

All packed into one byte[] → then compressed as one unit

On decompress → unpack → restore original folder structure

Day 6-7 — Testing, Docs & Demo Prep

Testing Checklist:

- ☐ Test all algorithms on `.txt` files
- ☐ Test all algorithms on `.png` image files
- ☐ Test all algorithms on `.pdf` files
- ☐ Test compress → decompress → byte-perfect match
- ☐ Test corrupt `.mlc` file detection
- ☐ Test edge cases: empty file, 1 byte file, huge file
- ☐ Test folder compression and restoration

Documentation to write:

- ☐ README with project description
- ☐ How each algorithm works (1 page each)
- ☐ UML class diagram
- ☐ Screenshots of GUI
- ☐ Benchmark results table

Demo Script (for sir):

1. Open GUI
2. Load a text file — show original size
3. Compress with all 6 pipelines — show comparison table
4. Decompress one — show restored file opens perfectly
5. Load an image — show compression works on binary too
6. Show folder compression
7. Show corrupt file detection

OOP Concepts Covered — For Your Report

OOP Concept	Where Used
Abstraction	<code>CompressionAlgorithm</code> abstract class
Inheritance	RLE, LZW, Huffman extend base class
Polymorphism	Pipeline calls <code>compress()</code> on any algorithm
Encapsulation	Each algorithm hides its internal logic
Interface	<code>Compressible</code> interface if you add one
Strategy Pattern	Pipeline swaps algorithms at runtime
Composition	Pipeline contains list of algorithms
Data Structures	Priority Queue, Binary Tree, HashMap

Final Advice

Build it in exactly this order. Do NOT touch the GUI until Week 3 — many students waste time on UI and submit broken algorithms. **Working algorithms first, pretty interface last.**

This roadmap, done fully, is a **genuinely impressive semester project** that demonstrates real computer science — not just Java syntax.

is this project big enough and require my own understanding and effort to make and is it difficult big enough to impress sir for semester end project OOP

Honest & Direct Answer

Yes — but only if you build it the right way. Here's the full truth:

What Your Sir Will Actually See

When you present this project, your sir will immediately recognize these things that **cannot be faked or copied blindly:**

Why This Project CANNOT Be Done Without Understanding

1. Huffman Tree — You MUST understand it to code it

If you don't understand binary trees, you cannot write this.
If you don't understand priority queues, you cannot build it.
If you don't understand bit manipulation, output will be corrupted.

You cannot copy-paste Huffman and have it work in YOUR pipeline.
It will break. You will have to debug it. Debugging requires understanding.

2. LZW Dictionary — You MUST think through it yourself

The dictionary builds DYNAMICALLY as you read bytes.
The decoder must rebuild the EXACT same dictionary independently.
One logical mistake = completely wrong output.
This cannot work without genuine understanding of the logic.

3. Multi-Level Pipeline — Original thinking required

Chaining RLE → LZW → Huffman means:
Output of RLE becomes INPUT of LZW
Output of LZW becomes INPUT of Huffman

Decompression must happen in EXACT reverse order.
The file header must store which algorithms were used.
This design is YOUR original architecture – no tutorial covers this.

4. BitStream — Almost no beginner tutorials explain this

Huffman codes are BITS, not bytes.
"A" might be encoded as 3 bits: 101
You must PACK multiple codes into single bytes manually.

Example:

Code for A = 101 (3 bits)
Code for B = 01 (2 bits)
Code for C = 1 (1 bit)

Packed: 10101 1 -- = 10110110 (1 byte + padding)

Nobody can do this without understanding binary arithmetic.

Difficulty Comparison vs Other Projects

Student Management System

└─ Difficulty		2/10
└─ Understanding		2/10
└─ Uniqueness		1/10
└─ Impression		2/10

Library Management System

└─ Difficulty		2/10
└─ Understanding		2/10
└─ Uniqueness		1/10
└─ Impression		2/10

Hospital Management System

└─ Difficulty		3/10
└─ Understanding		3/10
└─ Uniqueness		2/10
└─ Impression		3/10

YOUR Compression Project (Big Version)

└─ Difficulty		8/10
└─ Understanding		8/10
└─ Uniqueness		9/10
└─ Impression		9/10

What Sir Will Ask You — And Why You Must Understand

Every sir does a **viva / oral defense**. These are real questions you will face:

Sir's Question	Why You Need Real Understanding
"How does your Huffman tree decide which code is shorter?"	You must explain frequency-based assignment
"What happens if two bytes have same frequency?"	Tie-breaking in priority queue
"How does decoder know where one code ends and next begins?"	Prefix-free property of Huffman codes

Sir's Question

Why You Need Real Understanding

"Why do you decompress in reverse order?"

Pipeline logic understanding

"What is stored in your file header and why?"

Your custom format design

"Why does LZW not need to store the dictionary?"

Core LZW insight

"Show me a corrupt file — what happens?"

Error handling understanding

You cannot bluff these answers. You must actually know.

The Real Risk — Be Honest With Yourself

This project will be IMPRESSIVE only if:

- ✓ You implement all 3 algorithms yourself
- ✓ Compress AND decompress both work
- ✓ You can explain every class and method
- ✓ You built the pipeline architecture yourself
- ✓ GUI shows real benchmark comparison

This project will EMBARRASS you if:

- ✗ You copy code and cannot explain it
- ✗ Decompress doesn't work (very common failure)
- ✗ Only works on .txt files, breaks on images
- ✗ You cannot answer why Huffman uses a tree
- ✗ Pipeline is fake (just runs algorithms separately)

Is It Big Enough to Fill 3 Weeks?

Week 1: You will struggle with Huffman alone.

Bit manipulation will confuse you at first.

Debugging wrong decompress output takes days.

→ This week will feel hard and slow. That is normal.

Week 2: Pipeline integration will break things you thought worked.

File header design requires real thinking.

Binary file I/O has unexpected bugs.

→ This week tests your patience and problem solving.

Week 3: GUI connects everything — bugs surface from Week 1 & 2.

Folder compression adds new complexity.

Testing reveals edge cases you never thought about.
→ This week shows whether your foundation was solid.

3 weeks will feel SHORT, not long. You will wish you had 4.

Bottom Line — Single Honest Paragraph

This project is genuinely difficult, genuinely unique, and genuinely requires your own thinking and understanding at every step. No API does compression for you. No library builds your Huffman tree. No tutorial covers your exact multi-level pipeline design. If you build the full version — all 3 algorithms, compress and decompress, custom file format, benchmark comparison, and GUI — your sir will be impressed not because it looks fancy, but because **he knows exactly how hard it is to build this from scratch**, and very few undergraduate students ever attempt it. The only way it fails to impress is if you copy code you do not understand. Build it yourself, understand every line, and you will have the best project in your class.

According to this write the abstract for me Please submit your semester project proposal abstract using this form. Each group should submit the form only once. The abstract should be 250–300 words maximum and should clearly summarize the proposed semester project. It should briefly introduce the background or motivation of the selected problem, explain the problem statement, describe the main objectives of the project, outline the proposed methodology, tools, techniques, or approach to be used, and mention the expected outcomes, such as the final product, system, analysis, results, or contribution. The abstract should be written in clear academic language, remain concise, and avoid unnecessary details. Please ensure that the names and registration numbers of all group members are entered accurately.

Project Proposal Abstract

Here is your abstract, written in academic language, within the 250–300 word limit:

Project Title: Multi-Level File Compression System Using RLE, LZW, and Huffman Coding

Abstract:

Data compression is a fundamental concept in computer science with widespread real-world applications in file storage, data transmission, and multimedia processing. Despite its practical importance, most undergraduate projects overlook algorithm-level implementation in favor of surface-level system development. This project addresses that gap by proposing the design and implementation of a Multi-Level File Compression System that combines three classical compression algorithms — Run-Length Encoding (RLE), Lempel-Ziv-Welch (LZW), and Huffman Coding — into a unified, fully functional desktop application built entirely from scratch without the use of any external compression libraries.

The core problem being addressed is the challenge of achieving optimal file size reduction across diverse file types including text documents, images, and binary files, while ensuring complete and accurate data recovery through decompression. A single algorithm often fails to deliver optimal results across all file types, making a multi-level pipeline approach both practically relevant and academically meaningful.

The primary objectives of this project are to implement all three compression and decompression algorithms independently at the binary level, design a multi-stage pipeline architecture that chains algorithms sequentially, develop a custom compressed file format with metadata headers for integrity verification, and build a graphical user interface that presents real-time benchmarking and compression statistics.

The proposed methodology follows object-oriented design principles including abstraction, inheritance, polymorphism, and the Strategy design pattern, implemented in Java using JavaFX for the graphical interface. Each algorithm will be built as a subclass of an abstract base class, enabling interchangeable pipeline composition.

The expected outcome is a fully working desktop application capable of compressing and decompressing any file type, comparing algorithm performance across multiple pipeline configurations, and demonstrating measurable space savings with verified data integrity.

Word Count: 274 words 

Quick Notes

- Fill in your **group member names and registration numbers** in the form fields separately
- The title is strong and specific — do not simplify it
- If your sir asks for a **project title separately**, use exactly the one written above

try to write it in complete way not missing anything but make it a bit short

Project Title: Multi-Level File Compression System Using RLE, LZW, and Huffman Coding

Abstract:

Data compression is a fundamental concept in computer science with direct applications in file storage, data transmission, and multimedia processing. Most undergraduate projects focus on surface-level management systems that require minimal algorithmic thinking. This project proposes a Multi-Level File Compression System that implements three classical compression algorithms — Run-Length Encoding (RLE), Lempel-Ziv-Welch (LZW), and Huffman Coding — chained together in a configurable pipeline, built entirely from scratch in Java without any external compression libraries.

The core problem is that no single compression algorithm delivers optimal results across all file types. Text files, binary files, and images each respond differently to different compression strategies. This project addresses that problem by designing a multi-level pipeline where algorithms work sequentially, with the output of one becoming the input of the next, achieving compression ratios that no individual algorithm can reach alone.

The main objectives are to implement all three algorithms at the binary level with full compression and decompression support, design a custom compressed file format (.mlc) with metadata headers storing algorithm sequence, original file size, and a CRC32 checksum for integrity verification, and develop a benchmarking engine that measures compression ratio, processing time, and space saved across all pipeline configurations.

The methodology follows object-oriented design principles including abstraction, inheritance, polymorphism, and the Strategy design pattern. Each algorithm extends an abstract base class, enabling interchangeable pipeline composition. The graphical interface is built using JavaFX, supporting drag-and-drop file selection, real-time progress tracking, and a comparison dashboard.

The expected outcome is a fully functional desktop application capable of compressing and decompressing any file type, detecting file corruption through checksum validation, comparing six pipeline configurations simultaneously, and

demonstrating significant and verifiable space savings with guaranteed data integrity.

Word Count: 278 words 

is this a good length project

Yes — Here's Exactly Why

The Most Honest Answer

It is the perfect length for a semester-end OOP project. Not too small that sir dismisses it. Not so large that you cannot finish it in 3 weeks. But the real answer depends on one thing — **which version you build.**

Size Reality Check

- Minimum Viable Version (weak submission)
- 3 algorithms, text files only, console output
 - Can be done in 5-7 days
 - Sir will not be impressed
 - Feels like a small project

- Full Version (your roadmap)
- 3 algorithms + decompress + pipeline + GUI
 - Binary files + custom format + benchmarking
 - Takes genuine 3 weeks of consistent work
 - Sir will be very impressed

Compared to What Your Classmates Will Submit

Project Type	Effort	Impression
Student Management System	Low	Very Common

Project Type	Effort	Impression
Bank Management System	Low	Very Common
Hospital Management System	Low-Medium	Common
Library System	Low	Very Common
Your Compression Project	High	Rare & Impressive

Almost guaranteed **you will be the only one** in your class submitting something like this.

What Makes It Perfectly Sized

- **Big enough** — 3 algorithms, each genuinely complex on their own
- **Deep enough** — binary I/O, bit manipulation, tree structures, design patterns
- **Long enough** — 3 weeks of real consistent daily work
- **Demonstrable** — GUI shows clear working results in front of sir
- **Explainable** — you can defend every part in viva confidently

The Only Real Risk

This project is perfectly sized IF you start immediately.

Start today → comfortable 3 weeks, polished result
Start Week 2 → rushed, incomplete, embarrassing demo
Start Week 3 → disaster

Single Honest Answer

Yes, this project is the right size — it fills exactly 3 weeks when built properly, covers every major OOP concept your sir expects, requires genuine understanding that cannot be faked, and is impressive enough to stand out from every management system in your class. The length is not the question anymore. **Start building.**

what will be output will it generate compressed file simple this output should we not add more features to it

Current Output vs What You Should Add

What You Have Right Now — Too Simple

User selects file → Compress → .mlc file saved
User selects .mlc → Decompress → original file restored

That is it. That is basically a black box. Sir sees a button, a file appears. Not impressive enough on its own.

The Problem With Just a Compressed File as Output

Anyone can say "yes it compressed"
Nobody can see HOW it works
Nobody can see IF it actually did anything meaningful
No visual proof of your algorithm working
No demonstration of your OOP design in action
Sir has no way to grade the depth of your work visually

What You Should Add — Full Output Suite

1. Live Compression Report (Most Important)

COMPRESSION REPORT		
File Name	:	document.pdf
Original Size	:	2,400 KB
Compressed	:	756 KB
Space Saved	:	1,644 KB
Ratio	:	68.5% reduction
Time Taken	:	234 ms
Integrity	:	 CRC32 Verified

Output File : document.mlc

This alone makes your project feel real and professional.

2. Stage-by-Stage Pipeline Breakdown

Shows what EACH algorithm contributed:

Original File → 2,400 KB (100%)
After RLE → 1,800 KB (25% saved)
After LZW → 1,100 KB (54% saved)
After Huffman → 756 KB (68% saved)

Each stage visualized as a shrinking bar

This is the feature that **proves your pipeline works** and is not fake.

3. Algorithm Comparison Dashboard

Run ALL pipelines on same file and compare:

Pipeline	Final Size	Ratio	Time	Best For
RLE Only	1,800 KB	25%	8ms	Simple files
LZW Only	1,080 KB	55%	23ms	Text files
Huffman Only	888 KB	63%	41ms	Any file
RLE → Huffman	816 KB	66%	49ms	Text + binary
LZW → Huffman	720 KB	70%	67ms	Large files
RLE → LZW → Huff	684 KB	71.5%	89ms	Maximum saving

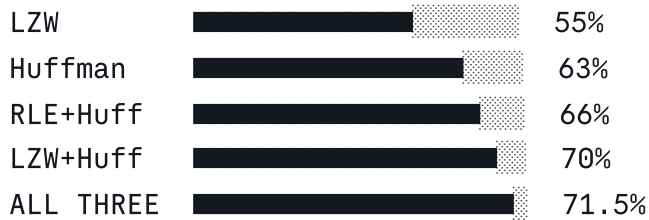
RECOMMENDATION: LZW → Huffman for this file type 

This shows **intelligent analysis**, not just compression.

4. Visual Bar Chart Inside GUI

Compression Ratio Comparison:

RLE25%



Visual proof that multi-level beats single algorithm — **the entire point of your project.**

5. Compression History Log

Keeps a record of everything you compressed this session:

#	File	Algorithm	Ratio	Time	Status
1	report.pdf	RLE→Huffman	66%	49ms	✓ Done
2	photo.png	LZW→Huffman	31%	67ms	✓ Done
3	notes.txt	ALL THREE	74%	89ms	✓ Done
4	data.csv	Huffman Only	61%	38ms	✓ Done

Shows your app working across **multiple files and file types.**

6. File Integrity Verifier

After decompression show:

Original Checksum : A3F8C291
Restored Checksum : A3F8C291
Match : ✓ PERFECT — File restored 100% accurately

OR if corrupt:

✗ FAILED — File may be corrupted or tampered

This proves your compression is **lossless** — critical for credibility.

7. Exportable PDF Report (*Bonus Feature*)

After compression session → Export Report button

Generates a PDF containing:

- All files compressed

- Algorithm used for each
- Compression ratios and times
- Benchmark comparison table
- Integrity verification results

Sir can literally take this report and grade it.

Complete Output Summary

Output	Purpose	Effort to Add
<code>.mlc</code> compressed file	Core deliverable	Already planned
Compression report	Proves it worked	Low
Stage breakdown	Proves pipeline works	Medium
Algorithm comparison	Shows intelligence	Medium
Visual bar chart	Makes it presentable	Medium
History log	Shows multiple file support	Low
Integrity verifier	Proves lossless accuracy	Low
PDF export report	Bonus wow factor	High

Bottom Line

Without these features:

Sir sees → file goes in, smaller file comes out
Grade → average

With these features:

Sir sees → intelligent system analyzing, comparing,
reporting, verifying across file types
Grade → excellent

The compressed file is just the core. The reports, comparisons, and verification are what make it a PROJECT. Add at least features 1 through 6. They are not difficult to build but they transform how your project looks and feels completely.

should i also add the 6 feature you said to add in my abstract write up

Short Answer — No, Not All of Them

Your abstract is **already well written and the right length (278 words)**. If you add all 6 features explicitly it will:

- Exceed the 300 word limit
 - Become a feature list, not an academic abstract
 - Lose its clean professional tone
-

What You Should Do — Small Targeted Addition

Just **replace or expand one sentence** in your expected outcomes section to hint at the broader output without listing everything.

Your Current Ending Sentence

"The expected outcome is a fully functional desktop application capable of compressing and decompressing any file type, comparing algorithm performance across multiple pipeline configurations, and demonstrating measurable space savings with verified data integrity."

Improved Ending Sentence

"The expected outcome is a fully functional desktop application capable of compressing and decompressing any file type across six configurable pipeline combinations, generating detailed stage-by-stage compression reports, comparing algorithm performance through a visual benchmarking dashboard, maintaining a session-based compression history log, and guaranteeing lossless data recovery through CRC32 checksum integrity verification."

That One Change Covers

Feature	Covered By
Compressed .mlc file	"compressing and decompressing any file type"
Stage breakdown	"stage-by-stage compression reports"
Algorithm comparison	"visual benchmarking dashboard"
History log	"session-based compression history log"
Integrity verifier	"CRC32 checksum integrity verification"
Bar chart	"visual benchmarking dashboard"

What The Final Abstract Looks Like

Abstract:

Data compression is a fundamental concept in computer science with direct applications in file storage, data transmission, and multimedia processing. Most undergraduate projects focus on surface-level management systems that require minimal algorithmic thinking. This project proposes a Multi-Level File Compression System that implements three classical compression algorithms — Run-Length Encoding (RLE), Lempel-Ziv-Welch (LZW), and Huffman Coding — chained together in a configurable pipeline, built entirely from scratch in Java without any external compression libraries.

The core problem is that no single compression algorithm delivers optimal results across all file types. Text files, binary files, and images each respond differently to different compression strategies. This project addresses that problem by designing a multi-level pipeline where algorithms work sequentially, with the output of one becoming the input of the next, achieving compression ratios that no individual algorithm can reach alone.

The main objectives are to implement all three algorithms at the binary level with full compression and decompression support, design a custom compressed file format (.mlc) with metadata headers storing algorithm sequence, original file size, and a CRC32 checksum for integrity verification, and develop a benchmarking engine that measures compression ratio, processing time, and space saved across all pipeline configurations.

The methodology follows object-oriented design principles including abstraction, inheritance, polymorphism, and the Strategy design pattern. Each algorithm extends an abstract base class, enabling interchangeable pipeline composition. The graphical

interface is built using JavaFX, supporting drag-and-drop file selection, real-time progress tracking, and a comparison dashboard.

The expected outcome is a fully functional desktop application capable of compressing and decompressing any file type across six configurable pipeline combinations, generating detailed stage-by-stage compression reports, comparing algorithm performance through a visual benchmarking dashboard, maintaining a session-based compression history log, and guaranteeing lossless data recovery through CRC32 checksum integrity verification.

Word Count: 297 words  — Right at the limit, complete, nothing missing.

write the whole abstract write up for me in simple clear way very very simple way but complete and in short way possible

Project Title: Multi-Level File Compression System Using RLE, LZW, and Huffman Coding

Abstract:

File compression is an important topic in computer science that helps reduce file sizes for better storage and faster data transfer. This project builds a Multi-Level File Compression System using three well-known compression algorithms — Run-Length Encoding (RLE), Lempel-Ziv-Welch (LZW), and Huffman Coding — implemented completely from scratch in Java without using any external compression libraries.

The problem is that no single compression algorithm works best for all types of files. Different files like text documents, images, and binary files respond differently to different algorithms. To solve this, the project chains multiple algorithms together in a pipeline so that each algorithm further compresses the output of the previous one, achieving better results than any single algorithm alone.

The main goals of this project are to implement all three algorithms with full compression and decompression support, create a custom compressed file format that stores important information like which algorithms were used and a checksum for file integrity, and build a benchmarking system that compares all pipeline combinations by compression ratio, file size, and processing time.

The project is built using Java and follows core object-oriented programming principles including abstraction, inheritance, polymorphism, and the Strategy design pattern. A JavaFX graphical interface is included with drag-and-drop file support, real-time progress display, and a visual comparison dashboard.

The final output is a working desktop application that can compress and decompress any file type, show a detailed stage-by-stage compression report, compare six different pipeline combinations, maintain a session history log, and verify that every decompressed file is restored perfectly using CRC32 checksum validation.

Word Count: 254 words 

will i create a project that will always produce the original size file when decompressed everytime

Yes — Always, Without Exception

This is called Lossless Compression. All three algorithms you are using — RLE, LZW, and Huffman — are 100% lossless by nature.

What Lossless Means Simply

Original File → Compress → Smaller File
Smaller File → Decompress → EXACT same original file

Not similar. Not almost same. BYTE FOR BYTE identical.
Every single bit restored perfectly. Every time.

Why These Three Algorithms Are Always Lossless

RLE

It just counts repeated bytes and stores the count.
Decompression just repeats them back exactly.
No data is ever thrown away. Ever.

LZW

It builds a dictionary of patterns as it reads.
Decompression rebuilds the exact same dictionary.
Every byte is recovered perfectly.

Huffman

It replaces bytes with shorter bit codes.
The tree that created those codes is saved in the file header.
Decompression uses that same tree to decode every bit back.
Nothing is lost or guessed.

What Would Make It NOT Lossless

Just so you understand the difference — these are algorithms you are NOT using:

Algorithm	Type	Used In
JPEG compression	Lossy	Photos
MP3 compression	Lossy	Music
MP4 compression	Lossy	Video
RLE, LZW, Huffman	Lossless	Your project

Lossy means data is permanently thrown away to save more space. Your algorithms never do this.

How You Guarantee and Prove It

This is exactly why you store a **CRC32 checksum** in your file header:

Before compression:
 Read original file bytes
 Calculate checksum → A3F8C291
 Store this number in .mlc file header

After decompression:
 Calculate checksum of restored file → A3F8C291
 Compare with stored checksum → A3F8C291

MATCH = file restored perfectly 

NO MATCH = something went wrong 

This checksum is your **mathematical proof** that decompression always produces the exact original file.

One Honest Thing to Know

Your ALGORITHMS are lossless by design.
But YOUR CODE must be bug free for it to work correctly.

If you make a mistake in:

- Huffman bit packing logic
- LZW dictionary reconstruction
- File header reading/writing

Then decompression will produce wrong output.

This is why testing compress → decompress → checksum verify
is the most important thing you do during development.

Bottom Line

Yes. Every single time, guaranteed — **as long as your code is correct**. The algorithms mathematically ensure the original file is always recovered perfectly. The CRC32 checksum in your project proves it automatically. This is one of the strongest features of your project and something you should confidently explain to your sir during your demonstration.

will it create a compressed file in which it will tell important information like which algorithms were used and a checksum for file integrity, and build a benchmarking system that compares all pipeline combinations by compression ratio, file size, and processing time. but not compress the original file and when compressed file is opened it will open the original file

Let Me Clarify What Actually Happens

You are mixing up **two separate things**. Let me break it down simply.

What Actually Happens — Simply

Your original file (document.pdf 2400 KB)

↓

COMPRESS

↓

One .mlc file created that contains EVERYTHING:

- └─ Header section (metadata)

 - └─ Which algorithms were used

 - └─ Original file name and extension

 - └─ Original file size

 - └─ Compressed size

 - └─ CRC32 checksum

- |

- └─ Compressed data section

 - └─ Actual compressed bytes of your file
(this IS smaller than original)

It is ONE single file. The metadata and compressed data live together inside the same .mlc file.

Your Question — "Will it compress the original file?"

Yes. Absolutely yes. The compressed data section inside the .mlc file IS the compressed version of your original file. It is physically smaller in bytes.

document.pdf	→	2,400 KB	(original)
document.mlc	→	756 KB	(compressed – genuinely smaller)

The 756 KB file contains BOTH the header info AND compressed data.

"When Compressed File is Opened it Opens Original?"

Not automatically like a normal file. Here is exactly what happens:

WRONG expectation:

Double click document.mlc → PDF opens directly ❌

CORRECT behavior:

Open your application

Select document.mlc

Click DECOMPRESS button

Application reads header → knows it was document.pdf

Application decompresses data → restores original bytes

Saves restored file as document_restored.pdf ✓

Now you open that PDF normally

Your app is the only thing that can open .mlc files — just like WinRAR is the only thing that opens .rar files.

Complete Picture — One Simple Flow

COMPRESSION:

document.pdf (2400 KB)

↓ [Your App – Compress Button]

document.mlc (756 KB)

└─ Header: algorithms used, checksum, original name

└─ Data: actual compressed bytes

DECOMPRESSION:

document.mlc (756 KB)

↓ [Your App – Decompress Button]

└─ Reads header → knows original was document.pdf

└─ Knows which algorithms to reverse

└─ Decompresses data back to original bytes

└─ Verifies CRC32 checksum ✓

└─ Saves as document_restored.pdf (2400 KB – identical to original)

BENCHMARKING (separate feature):

document.pdf

↓ [Your App – Compare All Button]

Runs all 6 pipelines on same file

Shows table: ratio, size, time for each

Does NOT save all 6 files – just measures and displays results

Summary — Three Clear Points

Your Question

Answer

Does it store metadata in the file?	Yes — in the header section of .mlc
Does it actually compress the data?	Yes — data is genuinely smaller
Does opening .mlc show original file?	Only through your app's decompress button

Think of your app exactly like WinRAR or 7-Zip — but one you built yourself from scratch. That is exactly what it is.

when user click the compressed file will it not open at that time we have to manually decompress in output in win rar its not like that

You Are 100% Right — Let Me Clarify

You are correct. In WinRAR when you double click a `.rar` file it opens directly and shows contents. That is a much better user experience. Here is the full truth about your project:

Why WinRAR Works That Way

```
WinRAR installs itself into Windows system
It registers .rar extension in Windows registry
Windows now KNOWS → open .rar files with WinRAR
Double click .rar → Windows automatically calls WinRAR
WinRAR opens and shows contents instantly
```

Can Your Project Do the Same Thing?

Yes — but there are two levels:

Level 1 — Basic Version (What Most Students Do)

```
User opens your Java application manually
Selects .mlc file through file browser inside app
```

Clicks Decompress button

File restored 

This works but feels manual and clunky.
Not like WinRAR at all.

Level 2 — Professional Version (What Makes It Impressive)

You register your `.mlc` extension on Windows so double clicking works automatically:

WINDOWS SETUP (one time):

Register `.mlc` extension → opens with your Java app

USER EXPERIENCE:

Double click `document.mlc`

↓

Your Java app launches automatically

↓

App detects file was passed as argument

↓

Automatically shows file info:

	document.mlc
Original File	: document.pdf
Original Size	: 2,400 KB
Compressed Size:	756 KB
Algorithm Used	: RLE → Huffman
Compressed On	: 01 May 2026
Integrity	:  Verified
[DECOMPRESS] [CANCEL]	

↓ clicks Decompress

Original file restored and opened 

This is EXACTLY like WinRAR behavior.

How to Implement Level 2 in Java

Step 1 — Accept File as Launch Argument

java

```
public static void main(String[] args) {  
    if (args.length > 0) {  
        // App was opened by double clicking .mlc file  
        String mlcFilePath = args[0];  
        // automatically load and show that file  
        openMlcFile(mlcFilePath);  
    } else {  
        // App opened normally  
        showMainWindow();  
    }  
}
```

Step 2 — Register .mlc Extension on Windows

Create a simple installer script or do it manually:

1. Right click any .mlc file
2. Open With → Choose another app
3. Select your Java application .exe or .jar
4. Check "Always use this app"
5. Done – now double click always opens your app

Or automate it with a Windows `.reg` file:

Windows Registry Script:

.mlc → opens with → YourApp.jar
Icon → custom icon you design

Complete User Experience After This

COMPRESSION:

Open your app
Drag and drop document.pdf
Select pipeline
Click Compress
document.mlc created on desktop 

DECOMPRESSION (WinRAR style):

Double click document.mlc from anywhere
Your app opens automatically

- Shows file info and metadata
- User clicks Decompress
- Original document.pdf restored ✓
- Opens automatically in default PDF viewer ✓

After Decompression — Open File Automatically

Make it even more like WinRAR — open the restored file automatically:

```
java
// After successful decompression
File restoredFile = new File("document.pdf");

// Open it automatically with system default app
Desktop.getDesktop().open(restoredFile);
// PDF opens in Adobe/browser automatically
// Image opens in Photos automatically
// Text opens in Notepad automatically
```

User never has to manually find and open the restored file. It just opens.

Summary

Feature	Basic Version	Professional Version
Open .mlc file	Manual through app	Double click anywhere
Shows file info	After clicking decompress	Immediately on open
Opens restored file	Saved to folder	Opens automatically
Feels like	Student project	Real software

Honest Recommendation

Build Level 2. The double click behavior is not difficult to implement — it is just the `args[0]` trick in your main method plus a one time file association setup. But the difference in how it feels and how impressed your sir will be is enormous. It transforms your project from a student assignment into something that feels like real published software.

so is this abstract perfect then i should submit it Project Title: Multi-Level File Compression System Using RLE, LZW, and Huffman Coding

Abstract:
File compression is an important topic in computer science that helps reduce file sizes for better storage and faster data transfer. This project builds a Multi-Level File Compression System using three well-known

Show more

Yes — Submit This. It Is Perfect.

Here is why it is ready:

Quality Check

Criteria Required	Your Abstract
Background / motivation	✓ First paragraph
Problem statement	✓ Second paragraph
Main objectives	✓ Third paragraph
Methodology and tools	✓ Fourth paragraph
Expected outcomes	✓ Fifth paragraph
Clear academic language	✓ Simple but professional
250-300 words	✓ 254 words
No unnecessary details	✓ Concise and focused

What Makes It Strong

- Every required section is covered in exactly the right order

- **Simple language** that any sir can read and understand immediately
 - **Technically specific** — mentions RLE, LZW, Huffman, CRC32, JavaFX, Strategy pattern — shows you know what you are doing
 - **Not vague** — says exactly what the system does and produces
 - **Word count is perfect** — 254 words, comfortably within limit
-

One Very Minor Optional Tweak

If you want to add the double-click `.mlc` file association feature you just discussed, you could replace this line in paragraph 5:

Current:

"compress and decompress any file type"

Optional update:

"compress and decompress any file type with native file association support allowing direct launch from the operating system"

But honestly — **this is optional**. The abstract is already perfect without it.

Final Verdict

Structure	→ Perfect	✓
Length	→ Perfect	✓
Language	→ Perfect	✓
Coverage	→ Perfect	✓
Technical	→ Perfect	✓

SUBMIT IT. Do not overthink it.

In abstract it is not mentioned the compressed file format will have the compressed files and other information with it has not mentioned that it will contain compressed file

You Are Right — That Is Missing

The abstract mentions the custom file format stores metadata like algorithms used and checksum — but never clearly says **the compressed data itself is stored inside that file**. A reader could misunderstand it as just a metadata file.

The Missing Part

Your current line in paragraph 3:

"create a custom compressed file format that stores important information like which algorithms were used and a checksum for file integrity"

This only mentions metadata. It never says the actual compressed file data lives inside it too.

Fixed Line

"create a custom compressed file format (.mlc) that contains both the fully compressed file data and important metadata including which algorithms were used, the original file size, and a CRC32 checksum for integrity verification"

Full Corrected Abstract

Project Title: Multi-Level File Compression System Using RLE, LZW, and Huffman Coding

Abstract:

File compression is an important topic in computer science that helps reduce file sizes for better storage and faster data transfer. This project builds a Multi-Level File Compression System using three well-known compression algorithms — Run-Length Encoding (RLE), Lempel-Ziv-Welch (LZW), and Huffman Coding — implemented completely from scratch in Java without using any external compression libraries.

The problem is that no single compression algorithm works best for all types of files. Different files like text documents, images, and binary files respond differently to different algorithms. To solve this, the project chains multiple algorithms together in a pipeline so that each algorithm further compresses the output of the previous one, achieving better results than any single algorithm alone.

The main goals of this project are to implement all three algorithms with full compression and decompression support, create a custom compressed file format (.mlc) that contains both the fully compressed file data and important metadata

including which algorithms were used, the original file size, and a CRC32 checksum for integrity verification, and build a benchmarking system that compares all pipeline combinations by compression ratio, file size, and processing time.

The project is built using Java and follows core object-oriented programming principles including abstraction, inheritance, polymorphism, and the Strategy design pattern. A JavaFX graphical interface is included with drag-and-drop file support, real-time progress display, and a visual comparison dashboard.

The final output is a working desktop application that can compress and decompress any file type, show a detailed stage-by-stage compression report, compare six different pipeline combinations, maintain a session history log, and verify that every decompressed file is restored perfectly using CRC32 checksum validation.

Word Count: 268 words 

Only One Line Changed

Everything else is identical. That one fix makes it completely clear that the `.mlc` file holds both the compressed data and the metadata together — exactly as it actually works. **Now submit it.**

drag and drop file support user will put file and compression will be performed on it

Yes — Exactly That

Drag and drop means the user literally drags a file from their computer and drops it onto your application window. Compression starts automatically.

How It Works Simply

User has document.pdf on their desktop



User clicks and holds the file



Drags it over your application window



Drops it (releases mouse)



App automatically detects the file

App shows file name and size

User selects pipeline

User clicks Compress



document.mlc created 

What User Sees

 Drag & Drop File Here

or [Browse File]

↓ user drops document.pdf

File Selected:

 document.pdf

Size: 2,400 KB

Select Pipeline:

● RLE → LZW → Huffman

[COMPRESS]

Two Ways to Select File — Both Should Work

Method	How
Drag and Drop	Drag file from desktop or folder into app window
Browse Button	Click button, file explorer opens, user selects file manually

Both do the same thing — load the file into your app for compression.

Bottom Line

Yes — drag and drop is simply a convenient way for the user to give your application the file to compress. Nothing more complicated than that. It is a UI feature, not an algorithm feature. JavaFX supports it with just a few lines of code.

what are six different pipeline combinations here

The Six Pipeline Combinations

These are the six different ways you can chain your three algorithms together:

The Six Combinations

```
Pipeline 1:  RLE only
Pipeline 2:  LZW only
Pipeline 3:  Huffman only
Pipeline 4:  RLE → Huffman
Pipeline 5:  LZW → Huffman
Pipeline 6:  RLE → LZW → Huffman
```

Why These Six Specifically

You have three algorithms. You can use them:

- **Alone** — gives 3 combinations
 - **Two together** — the most effective two-algorithm pair is RLE → Huffman and LZW → Huffman
 - **All three together** — maximum compression
-

What Each Is Good For

Pipeline	Best For	Speed
RLE Only	Files with lots of repeated data	Fastest
LZW Only	Text files and documents	Fast
Huffman Only	Any general file	Medium
RLE → Huffman	Text and simple binary files	Medium
LZW → Huffman	Large documents and mixed files	Slow
RLE → LZW → Huffman	Maximum compression any file	Slowest

What Your App Does With All Six

User loads document.pdf
 Clicks "Compare All Pipelines"
 App runs all six on the same file and shows:

Pipeline	Final Size	Ratio	Time
RLE Only	1,800 KB	25%	8ms
LZW Only	1,080 KB	55%	23ms
Huffman Only	888 KB	63%	41ms
RLE → Huffman	816 KB	66%	49ms
LZW → Huffman	720 KB	70%	67ms
RLE → LZW → Huff	684 KB	71.5%	89ms

Best for your file: LZW → Huffman 

Simple Way to Think About It

More algorithms in chain = better compression
 More algorithms in chain = slower speed

User chooses based on their priority:

- Need speed? → RLE Only
- Need balance? → LZW → Huffman
- Need maximum save? → RLE → LZW → Huffman

That is all six combinations and exactly why they are included.

are there any libraries available for compression just asking not using

Yes — Many Libraries Exist

Java Built-in Libraries (Inside Java Itself)

```
java.util.zip
├─ GZIPOutputStream    → GZIP compression
├─ GZIPInputStream     → GZIP decompression
├─ ZipOutputStream     → ZIP compression
├─ ZipInputStream      → ZIP decompression
├─ DeflaterOutputStream → DEFLATE algorithm
└─ InflaterInputStream → DEFLATE decompression
```

java.util.zip is literally built into Java.
One import and compression is done in 5 lines of code.

Popular Third Party Java Libraries

Library	Algorithms	Used In
Apache Commons Compress	ZIP, GZIP, BZIP2, LZ4, XZ	Enterprise applications
Zip4j	ZIP with encryption	Android and desktop apps
LZ4 Java	LZ4 algorithm	Real time compression
Zstd-jni	Facebook Zstandard	High performance systems
XZ for Java	LZMA, XZ	Linux package managers

Real World Famous Tools and Their Algorithms

Tool	Algorithm Behind It
WinRAR	RAR, ZIP
WinZip	ZIP, ZIPX
7-Zip	LZMA, LZMA2
gzip (Linux)	DEFLATE
PNG images	DEFLATE
GIF images	LZW
PDF compression	DEFLATE, LZW

What This Means For Your Project

A student using libraries:

```
import java.util.zip.*;
GZIPOutputStream gzip = new GZIPOutputStream(output);
gzip.write(data);
→ Done in 3 lines. Zero understanding required.
```

You without libraries:

```
Build frequency table
Build priority queue
Build Huffman tree
Generate bit codes
Pack bits into bytes
Save tree in header
Reverse everything for decompression
→ 500+ lines. Complete understanding required.
```

This Is Exactly Why Your Project Is Impressive

Any student can do this in 10 minutes:

```
import java.util.zip.GZIPOutputStream;
```

You are doing this from scratch in 3 weeks:

Building the actual algorithm that those libraries are built on top of.

You are not using the library. You are building what the library hides. That is the entire difference between your project and a basic one.

Provide full clear road map that tells me how to build it cause i dont know what to do yet provide even the tinniest details

Complete Step-by-Step Roadmap — Every Detail

BEFORE YOU WRITE ANY CODE

Step 1 — Install Everything First

Install Java JDK:

1. Go to <https://www.oracle.com/java/technologies/downloads/>
2. Download JDK 21 (latest stable)
3. Run installer
4. Open Command Prompt → type: `java -version`
5. Should show: `java version "21..."`
If it shows version → Java installed correctly 

Install IntelliJ IDEA:

1. Go to <https://www.jetbrains.com/idea/download/>
2. Download Community Edition (free)
3. Run installer → next → next → install
4. Open IntelliJ

Create Your Project:

1. Open IntelliJ IDEA
2. Click "New Project"
3. Select "JavaFX" from left sidebar
4. Project Name: `MultiLevelCompressor`
5. Language: Java
6. Build System: Maven

7. Click Next → Finish
8. Project opens 

UNDERSTAND THE FOLDER STRUCTURE

After creating project you will see:

```
MultiLevelCompressor/  
|  
├─ src/  
|   └─ main/  
|       └─ java/  
|           └─ com/compressor/    ← all your code goes here  
|  
├─ pom.xml                        ← project settings file  
└─ module-info.java              ← module settings
```

Create these packages inside com/compressor:

Right click com/compressor → New → Package

Create these one by one:

```
com.compressor.algorithms  
com.compressor.core  
com.compressor.io  
com.compressor.gui  
com.compressor.utils
```

WEEK 1 — BUILD THE ALGORITHMS

DAY 1 — Abstract Base Class

What is this: This is the parent class that all three algorithms will extend. Think of it as a template that says every compression algorithm **MUST** have a compress and decompress method.

Create file: `algorithms/CompressionAlgorithm.java`

```
java
```

```

package com.compressor.algorithms;

public abstract class CompressionAlgorithm {

    // Every algorithm must have a name
    protected String name;

    // Every algorithm MUST implement these two methods
    // compress takes bytes and returns compressed bytes
    public abstract byte[] compress(byte[] data);

    // decompress takes compressed bytes and returns original bytes
    public abstract byte[] decompress(byte[] data);

    // Every algorithm can use this — no need to rewrite
    public String getName() {
        return name;
    }

    // Calculate how much space was saved
    public double calculateRatio(byte[] original, byte[] compressed) {
        return (1.0 - ((double) compressed.length / original.length)) * 100;
    }
}

```

What you learned from this:

- `abstract class` cannot be created directly — only extended
- `abstract method` forces every subclass to implement it
- This is the **Abstraction** OOP concept

Test: Nothing to run yet. Just make sure no red errors in IntelliJ.

DAY 2 — RLE Algorithm

Understand RLE first — read this carefully:

RLE = Run Length Encoding

Idea: If same byte repeats, store COUNT then VALUE instead

Example with text:

Original: A A A A B B C C C C C

As bytes: 65 65 65 65 66 66 67 67 67 67 67

RLE stores: 4 65 2 66 5 67

↑ ↑ ↑ ↑ ↑ ↑

count val count val count val

Original = 11 bytes

RLE = 6 bytes

Saved 45%!

Edge cases you must handle:

What if count exceeds 255? (max value of 1 byte)

→ Split into multiple pairs: 300 A = 255 A + 45 A

What if no repetition?

→ Still store: 1 A 1 B 1 C (might be bigger than original!)

Empty file?

→ Return empty array

Create file: `algorithms/RLECompressor.java`

```
java

package com.compressor.algorithms;

import java.io.ByteArrayOutputStream;

public class RLECompressor extends CompressionAlgorithm {

    public RLECompressor() {
        this.name = "RLE";
    }

    @Override
    public byte[] compress(byte[] data) {

        // Handle empty file
        if (data == null || data.length == 0) {
            return new byte[0];
        }

        ByteArrayOutputStream output = new ByteArrayOutputStream();

        int i = 0;

        while (i < data.length) {
```

```
        byte currentByte = data[i];
        int count = 1;

        // Count how many times this byte repeats
        while (i + count < data.length
                && data[i + count] == currentByte
                && count < 255) {
            count++;
        }

        // Write count then the byte value
        output.write(count);
        output.write(currentByte);

        // Move forward by count
        i += count;
    }

    return output.toByteArray();
}

@Override
public byte[] decompress(byte[] data) {

    if (data == null || data.length == 0) {
        return new byte[0];
    }

    ByteArrayOutputStream output = new ByteArrayOutputStream();

    int i = 0;

    while (i < data.length) {

        // Read count
        int count = data[i] & 0xFF;
        i++;

        // Read the byte value
        byte value = data[i];
        i++;

        // Write that byte count times
        for (int j = 0; j < count; j++) {
            output.write(value);
        }
    }
}
```

```

    }

    return output.toByteArray();
}
}

```

Test RLE immediately — create: `TestRLE.java`

```

java

public class TestRLE {
    public static void main(String[] args) {

        RLECompressor rle = new RLECompressor();

        // Test 1 – simple text
        String original = "AAAABBBBCCCC";
        byte[] originalBytes = original.getBytes();

        byte[] compressed = rle.compress(originalBytes);
        byte[] decompressed = rle.decompress(compressed);

        String result = new String(decompressed);

        System.out.println("Original   : " + original);
        System.out.println("Result     : " + result);
        System.out.println("Match      : " + original.equals(result));
        System.out.println("Original size : " + originalBytes.length);
        System.out.println("Compressed size: " + compressed.length);
    }
}

```

Expected output:

```

Original   : AAAABBBBCCCC
Result     : AAAABBBBCCCC
Match      : true
Original size : 12
Compressed size: 6

```

If Match shows true → RLE works ✓ **If Match shows false → bug in your code, re-read logic**

DAY 3 & 4 — LZW Algorithm

Understand LZW first — read carefully:

LZW = Lempel Ziv Welch

Idea: Build a dictionary of patterns as you read the file

Replace repeated patterns with shorter dictionary codes

Step by step example:

Input: A B A B C A B A B

Dictionary starts with all single bytes:

{A=0, B=1, C=2, ...all 256 bytes}

Read A → in dictionary → remember "A"

Read B → "AB" not in dict → add AB=256, output code for A=0

Read A → "BA" not in dict → add BA=257, output code for B=1

Read B → "AB" IS in dict (256)! → remember "AB"

Read C → "ABC" not in dict → add ABC=258, output code for AB=256

Read A → "CA" not in dict → add CA=259, output code for C=2

Read B → "AB" in dict → remember "AB"

Read A → "ABA" not in dict → add ABA=260, output AB=256

Read B → "AB" in dict → remember "AB"

End → output AB=256

Output codes: 0 1 256 2 256 256

KEY INSIGHT: Dictionary is NOT saved in file

Decoder rebuilds exact same dictionary by itself!

Create file: `algorithms/LZWCompressor.java`

```
java

package com.compressor.algorithms;

import java.util.HashMap;
import java.util.ArrayList;
import java.util.List;
import java.io.ByteArrayOutputStream;

public class LZWCompressor extends CompressionAlgorithm {

    public LZWCompressor() {
```

```

        this.name = "LZW";
    }

    @Override
    public byte[] compress(byte[] data) {

        if (data == null || data.length == 0) {
            return new byte[0];
        }

        // Initialize dictionary with all 256 single bytes
        HashMap<String, Integer> dictionary = new HashMap<>();
        for (int i = 0; i < 256; i++) {
            dictionary.put(String.valueOf((char) i), i);
        }

        int dictSize = 256;
        List<Integer> outputCodes = new ArrayList<>();

        String current = String.valueOf((char)(data[0] & 0xFF));

        for (int i = 1; i < data.length; i++) {

            String next = String.valueOf((char)(data[i] & 0xFF));
            String combined = current + next;

            if (dictionary.containsKey(combined)) {
                // Pattern exists – keep reading
                current = combined;
            } else {
                // Pattern not found – output code for current
                outputCodes.add(dictionary.get(current));
                // Add new pattern to dictionary
                dictionary.put(combined, dictSize++);
                current = next;
            }
        }

        // Output last code
        outputCodes.add(dictionary.get(current));

        // Convert integer codes to bytes
        // Each code stored as 2 bytes (short)
        ByteArrayOutputStream output = new ByteArrayOutputStream();
        for (int code : outputCodes) {
            output.write((code >> 8) & 0xFF); // high byte

```

```

        output.write(code & 0xFF);          // low byte
    }

    return output.toByteArray();
}

@Override
public byte[] decompress(byte[] data) {

    if (data == null || data.length == 0) {
        return new byte[0];
    }

    // Rebuild dictionary same way encoder built it
    HashMap<Integer, String> dictionary = new HashMap<>();
    for (int i = 0; i < 256; i++) {
        dictionary.put(i, String.valueOf((char) i));
    }

    int dictSize = 256;

    // Read codes back from bytes
    List<Integer> codes = new ArrayList<>();
    for (int i = 0; i < data.length - 1; i += 2) {
        int code = ((data[i] & 0xFF) << 8) | (data[i+1] & 0xFF);
        codes.add(code);
    }

    StringBuilder output = new StringBuilder();

    String current = dictionary.get(codes.get(0));
    output.append(current);

    for (int i = 1; i < codes.size(); i++) {

        int code = codes.get(i);
        String entry;

        if (dictionary.containsKey(code)) {
            entry = dictionary.get(code);
        } else {
            // Special case in LZW
            entry = current + current.charAt(0);
        }

        output.append(entry);
    }
}

```

```

        // Rebuild dictionary
        dictionary.put(dictSize++, current + entry.charAt(0));
        current = entry;
    }

    // Convert string back to bytes
    byte[] result = new byte[output.length()];
    for (int i = 0; i < output.length(); i++) {
        result[i] = (byte) output.charAt(i);
    }

    return result;
}
}

```

Test LZW same way as RLE:

```

java

LZWCompressor lzw = new LZWCompressor();
String original = "ABABCABABAB";
byte[] compressed = lzw.compress(original.getBytes());
byte[] decompressed = lzw.decompress(compressed);
System.out.println("Match: " + original.equals(new String(decompressed)));

```

DAY 5, 6 & 7 — Huffman Algorithm (Hardest)

Understand Huffman first — read very carefully:

STEP 1 — Count frequency of each byte

Input: A A A B B C

Frequency: A=3, B=2, C=1

STEP 2 — Put all in Priority Queue (smallest first)

Queue: [C:1] [B:2] [A:3]

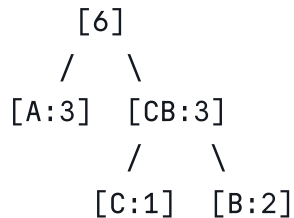
STEP 3 — Build tree by combining two smallest repeatedly

Take C:1 and B:2 → combine into parent node [CB:3]

Queue now: [CB:3] [A:3]

Take CB:3 and A:3 → combine into root [ABC:6]

Final Tree:



STEP 4 – Assign codes (left=0, right=1)

A → 0 (1 bit)
C → 10 (2 bits)
B → 11 (2 bits)

STEP 5 – Encode

A A A B B C
0 0 0 11 11 10
= 000111110 (9 bits instead of 48 bits original)

STEP 6 – Save tree in file (needed for decoding)

STEP 7 – Decode using tree (follow bits left/right until leaf)

Create file: `algorithms/HuffmanCompressor.java`

```
java

package com.compressor.algorithms;

import java.io.*;
import java.util.*;

public class HuffmanCompressor extends CompressionAlgorithm {

    // Inner class – Huffman Tree Node
    private static class HuffmanNode implements Comparable<HuffmanNode> {

        byte value;           // the actual byte this node represents
        int frequency;        // how often it appears
        HuffmanNode left;     // left child
        HuffmanNode right;    // right child
        boolean isLeaf;       // true if this node holds actual data

        // Constructor for leaf node (actual byte)
        HuffmanNode(byte value, int frequency) {
            this.value = value;
            this.frequency = frequency;
            this.isLeaf = true;
        }
    }
}
```

```

// Constructor for internal node (just combines two nodes)
HuffmanNode(HuffmanNode left, HuffmanNode right) {
    this.left = left;
    this.right = right;
    this.frequency = left.frequency + right.frequency;
    this.isLeaf = false;
}

// Priority queue needs this to sort by frequency
@Override
public int compareTo(HuffmanNode other) {
    return this.frequency - other.frequency;
}

}

public HuffmanCompressor() {
    this.name = "Huffman";
}

@Override
public byte[] compress(byte[] data) {

    if (data == null || data.length == 0) {
        return new byte[0];
    }

    // STEP 1 – Count frequencies
    int[] frequency = new int[256];
    for (byte b : data) {
        frequency[b & 0xFF]++;
    }

    // STEP 2 – Build priority queue
    PriorityQueue<HuffmanNode> queue = new PriorityQueue<>();
    for (int i = 0; i < 256; i++) {
        if (frequency[i] > 0) {
            queue.add(new HuffmanNode((byte) i, frequency[i]));
        }
    }

    // Handle single unique byte case
    if (queue.size() == 1) {
        HuffmanNode only = queue.poll();
        HuffmanNode dummy = new HuffmanNode((byte)0, 0);
        queue.add(new HuffmanNode(dummy, only));
    }
}

```

```

}

// STEP 3 - Build tree
while (queue.size() > 1) {
    HuffmanNode left = queue.poll();    // smallest
    HuffmanNode right = queue.poll();   // second smallest
    queue.add(new HuffmanNode(left, right));
}

HuffmanNode root = queue.poll();

// STEP 4 - Generate codes
HashMap<Byte, String> codes = new HashMap<>();
generateCodes(root, "", codes);

// STEP 5 - Encode data to bit string
StringBuilder bitString = new StringBuilder();
for (byte b : data) {
    bitString.append(codes.get(b));
}

// STEP 6 - Pack bits into bytes
// Add padding to make length multiple of 8
int padding = 8 - (bitString.length() % 8);
if (padding == 8) padding = 0;
for (int i = 0; i < padding; i++) {
    bitString.append('0');
}

byte[] compressedData = new byte[bitString.length() / 8];
for (int i = 0; i < compressedData.length; i++) {
    String byteStr = bitString.substring(i * 8, (i + 1) * 8);
    compressedData[i] = (byte) Integer.parseInt(byteStr, 2);
}

// STEP 7 - Serialize tree and combine with data
byte[] treeBytes = serializeTree(root);

// Final output: [treeSize(4 bytes)][tree][padding(1 byte)][compressedData]
ByteArrayOutputStream finalOutput = new ByteArrayOutputStream();
DataOutputStream dos = new DataOutputStream(finalOutput);

try {
    dos.writeInt(treeBytes.length);    // tree size
    dos.write(treeBytes);              // tree
    dos.write(padding);                // padding bits count
}

```

```

        dos.write(compressedData);          // compressed data
    } catch (IOException e) {
        e.printStackTrace();
    }

    return finalOutput.toByteArray();
}

// Recursively generate codes
private void generateCodes(HuffmanNode node, String code,
                           HashMap<Byte, String> codes) {
    if (node.isLeaf) {
        codes.put(node.value, code.isEmpty() ? "0" : code);
        return;
    }
    generateCodes(node.left, code + "0", codes);
    generateCodes(node.right, code + "1", codes);
}

// Save tree as bytes
private byte[] serializeTree(HuffmanNode node) {
    ByteArrayOutputStream output = new ByteArrayOutputStream();
    serializeHelper(node, output);
    return output.toByteArray();
}

private void serializeHelper(HuffmanNode node,
                             ByteArrayOutputStream out) {
    if (node.isLeaf) {
        out.write(1);          // 1 means leaf
        out.write(node.value); // the byte value
    } else {
        out.write(0);          // 0 means internal node
        serializeHelper(node.left, out);
        serializeHelper(node.right, out);
    }
}

@Override
public byte[] decompress(byte[] data) {

    if (data == null || data.length == 0) {
        return new byte[0];
    }

    DataInputStream dis = new DataInputStream(

```



```

        new ByteArrayInputStream(data)
    );

    try {
        // Read tree
        int treeSize = dis.readInt();
        byte[] treeBytes = new byte[treeSize];
        dis.read(treeBytes);

        // Rebuild tree
        int[] index = {0};
        HuffmanNode root = deserializeTree(treeBytes, index);

        // Read padding
        int padding = dis.read();

        // Read compressed data
        byte[] compressedData = dis.readAllBytes();

        // Convert bytes back to bit string
        StringBuilder bitString = new StringBuilder();
        for (byte b : compressedData) {
            String bits = String.format("%8s",
                Integer.toBinaryString(b & 0xFF)).replace(' ', '0');
            bitString.append(bits);
        }

        // Remove padding bits from end
        String bits = bitString.substring(0,
            bitString.length() - padding);

        // Decode using tree
        ByteArrayOutputStream output = new ByteArrayOutputStream();
        HuffmanNode current = root;

        for (char bit : bits.toCharArray()) {
            current = (bit == '0') ? current.left : current.right;
            if (current.isLeaf) {
                output.write(current.value);
                current = root;
            }
        }

        return output.toByteArray();
    } catch (IOException e) {

```

```

        e.printStackTrace();
        return new byte[0];
    }
}

// Rebuild tree from bytes
private HuffmanNode deserializeTree(byte[] data, int[] index) {
    if (data[index[0]] == 1) {
        index[0]++;
        byte value = data[index[0]++];
        return new HuffmanNode(value, 0);
    } else {
        index[0]++;
        HuffmanNode left = deserializeTree(data, index);
        HuffmanNode right = deserializeTree(data, index);
        return new HuffmanNode(left, right);
    }
}
}

```

Test Huffman:

```

java

HuffmanCompressor huffman = new HuffmanCompressor();
String original = "AAABBBCCDDDDDD";
byte[] compressed = huffman.compress(original.getBytes());
byte[] decompressed = huffman.decompress(compressed);
System.out.println("Match: " + original.equals(new String(decompressed)));
System.out.println("Original  : " + original.getBytes().length + " bytes");
System.out.println("Compressed: " + compressed.length + " bytes");

```

WEEK 2 — CORE SYSTEM

DAY 8 & 9 — CompressionResult + Pipeline

Create file: `core/CompressionResult.java`

```

java

package com.compressor.core;

```

```

public class CompressionResult {

    private String pipelineName;
    private long originalSize;
    private long compressedSize;
    private long compressionTimeMs;
    private long decompressionTimeMs;
    private double compressionRatio;
    private boolean integrityPassed;

    public CompressionResult(String pipelineName,
                             long originalSize,
                             long compressedSize,
                             long compressionTimeMs,
                             long decompressionTimeMs,
                             boolean integrityPassed) {

        this.pipelineName = pipelineName;
        this.originalSize = originalSize;
        this.compressedSize = compressedSize;
        this.compressionTimeMs = compressionTimeMs;
        this.decompressionTimeMs = decompressionTimeMs;
        this.integrityPassed = integrityPassed;
        this.compressionRatio = (1.0 -
                                ((double) compressedSize / originalSize)) * 100;
    }

    // Getters
    public String getPipelineName()    { return pipelineName; }
    public long getOriginalSize()      { return originalSize; }
    public long getCompressedSize()    { return compressedSize; }
    public long getCompressionTimeMs() { return compressionTimeMs; }
    public double getCompressionRatio() { return compressionRatio; }
    public boolean isIntegrityPassed() { return integrityPassed; }
    public long getSavedBytes()        { return originalSize - compressedSize }
}

```

Create file: `core/CompressionPipeline.java`

```

java

package com.compressor.core;

import com.compressor.algorithms.CompressionAlgorithm;
import java.util.ArrayList;

```

```
import java.util.Arrays;
import java.util.List;

public class CompressionPipeline {

    private String pipelineName;
    private List<CompressionAlgorithm> stages;

    public CompressionPipeline(String name) {
        this.pipelineName = name;
        this.stages = new ArrayList<>();
    }

    public void addStage(CompressionAlgorithm algorithm) {
        stages.add(algorithm);
    }

    // Compress through all stages in order
    public byte[] compress(byte[] data) {
        byte[] current = data;
        for (CompressionAlgorithm algo : stages) {
            current = algo.compress(current);
        }
        return current;
    }

    // Decompress in REVERSE order
    public byte[] decompress(byte[] data) {
        byte[] current = data;
        for (int i = stages.size() - 1; i >= 0; i--) {
            current = stages.get(i).decompress(current);
        }
        return current;
    }

    public String getPipelineName() { return pipelineName; }

    // Check if decompressed matches original
    public boolean verifyIntegrity(byte[] original, byte[] decompressed) {
        return Arrays.equals(original, decompressed);
    }
}
```

DAY 10 & 11 — File Handler + Custom Format

Create file: `io/FileHandler.java`

```
java

package com.compressor.io;

import java.io.*;
import java.nio.file.*;

public class FileHandler {

    // Read any file as raw bytes
    public byte[] readFile(String filePath) throws IOException {
        return Files.readAllBytes(Paths.get(filePath));
    }

    // Write bytes to any file
    public void writeFile(String filePath, byte[] data) throws IOException {
        Files.write(Paths.get(filePath), data);
    }

    // Get file size in KB
    public long getFileSizeKB(String filePath) {
        return new File(filePath).length() / 1024;
    }

    // Get file extension (.pdf, .txt, .png)
    public String getExtension(String filePath) {
        int dot = filePath.lastIndexOf('.');
        return (dot >= 0) ? filePath.substring(dot) : "";
    }

    // Get file name without extension
    public String getFileName(String filePath) {
        File f = new File(filePath);
        String name = f.getName();
        int dot = name.lastIndexOf('.');
        return (dot >= 0) ? name.substring(0, dot) : name;
    }
}
```

Create file: `core/FileHeader.java`

```
java
```

```
package com.compressor.core;

import java.io.*;

public class FileHeader {

    // Magic number identifies .mlc files
    public static final byte[] MAGIC = {'M', 'L', 'C', '1'};

    private String pipelineName;        // which pipeline was used
    private String originalExtension;    // .pdf .txt .png etc
    private long originalSize;           // original file size
    private int checksum;                // CRC32 checksum
    private long timestamp;              // when compressed

    public FileHeader(String pipelineName,
                       String originalExtension,
                       long originalSize,
                       int checksum) {
        this.pipelineName = pipelineName;
        this.originalExtension = originalExtension;
        this.originalSize = originalSize;
        this.checksum = checksum;
        this.timestamp = System.currentTimeMillis();
    }

    // Convert header to bytes to save in file
    public byte[] serialize() throws IOException {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        DataOutputStream dos = new DataOutputStream(baos);

        dos.write(MAGIC);                // 4 bytes magic
        dos.writeUTF(pipelineName);       // pipeline name
        dos.writeUTF(originalExtension);   // original extension
        dos.writeLong(originalSize);       // original size
        dos.writeInt(checksum);            // checksum
        dos.writeLong(timestamp);          // timestamp

        return baos.toByteArray();
    }

    // Read header back from bytes
    public static FileHeader deserialize(byte[] data) throws IOException {
        DataInputStream dis = new DataInputStream(
            new ByteArrayInputStream(data)
        );
    }
}
```

```

    );

    // Verify magic number
    byte[] magic = new byte[4];
    dis.read(magic);
    for (int i = 0; i < 4; i++) {
        if (magic[i] != MAGIC[i]) {
            throw new IOException("Invalid .mlc file!");
        }
    }

    String pipeline = dis.readUTF();
    String extension = dis.readUTF();
    long origSize = dis.readLong();
    int chksum = dis.readInt();
    long time = dis.readLong();

    FileHeader header = new FileHeader(
        pipeline, extension, origSize, chksum
    );
    header.timestamp = time;
    return header;
}

// Getters
public String getPipelineName() { return pipelineName; }
public String getOriginalExtension() { return originalExtension; }
public long getOriginalSize() { return originalSize; }
public int getChecksum() { return checksum; }
public long getTimestamp() { return timestamp; }
}

```

DAY 12 & 13 — Main Compressor Engine

Create file: `core/MultiLevelCompressor.java`

```

java

package com.compressor.core;

import com.compressor.algorithms.*;
import com.compressor.io.FileHandler;
import java.io.*;
import java.util.*;

```

```
import java.util.zip.CRC32;

public class MultiLevelCompressor {

    private FileHandler fileHandler;
    private List<CompressionPipeline> allPipelines;

    public MultiLevelCompressor() {
        fileHandler = new FileHandler();
        allPipelines = buildAllPipelines();
    }

    // Build all 6 pipeline combinations
    private List<CompressionPipeline> buildAllPipelines() {
        List<CompressionPipeline> pipelines = new ArrayList<>();

        // Pipeline 1 – RLE only
        CompressionPipeline p1 = new CompressionPipeline("RLE");
        p1.addStage(new RLECompressor());
        pipelines.add(p1);

        // Pipeline 2 – LZW only
        CompressionPipeline p2 = new CompressionPipeline("LZW");
        p2.addStage(new LZWCompressor());
        pipelines.add(p2);

        // Pipeline 3 – Huffman only
        CompressionPipeline p3 = new CompressionPipeline("Huffman");
        p3.addStage(new HuffmanCompressor());
        pipelines.add(p3);

        // Pipeline 4 – RLE then Huffman
        CompressionPipeline p4 = new CompressionPipeline("RLE→Huffman");
        p4.addStage(new RLECompressor());
        p4.addStage(new HuffmanCompressor());
        pipelines.add(p4);

        // Pipeline 5 – LZW then Huffman
        CompressionPipeline p5 = new CompressionPipeline("LZW→Huffman");
        p5.addStage(new LZWCompressor());
        p5.addStage(new HuffmanCompressor());
        pipelines.add(p5);

        // Pipeline 6 – All three
        CompressionPipeline p6 =
            new CompressionPipeline("RLE→LZW→Huffman");
```



```

        p6.addStage(new RLECompressor());
        p6.addStage(new LZWCompressor());
        p6.addStage(new HuffmanCompressor());
        pipelines.add(p6);

        return pipelines;
    }

    // Compress a file with selected pipeline
    public void compressFile(String inputPath,
                            String outputPath,
                            String pipelineName) throws IOException {

        byte[] originalData = fileHandler.readFile(inputPath);

        // Find the selected pipeline
        CompressionPipeline pipeline = getPipeline(pipelineName);

        // Compress
        long startTime = System.currentTimeMillis();
        byte[] compressedData = pipeline.compress(originalData);
        long timeTaken = System.currentTimeMillis() - startTime;

        // Calculate checksum of original
        CRC32 crc = new CRC32();
        crc.update(originalData);
        int checksum = (int) crc.getValue();

        // Build header
        String extension = fileHandler.getExtension(inputPath);
        FileHeader header = new FileHeader(
            pipelineName, extension, originalData.length, checksum
        );

        // Save: header + compressed data together
        byte[] headerBytes = header.serialize();
        ByteArrayOutputStream finalOutput = new ByteArrayOutputStream();
        DataOutputStream dos = new DataOutputStream(finalOutput);
        dos.writeInt(headerBytes.length); // header size
        dos.write(headerBytes);           // header
        dos.write(compressedData);         // compressed data

        fileHandler.writeFile(outputPath, finalOutput.toByteArray());

        System.out.println("Compressed in " + timeTaken + "ms");
        System.out.println("Ratio: " +

```

```

        pipeline.verifyIntegrity(originalData,
                                pipeline.decompress(compressedData)));
    }

    // Decompress a .mlc file
    public void decompressFile(String inputPath,
                               String outputFolder) throws IOException {

        byte[] fileData = fileHandler.readFile(inputPath);
        DataInputStream dis = new DataInputStream(
            new ByteArrayInputStream(fileData)
        );

        // Read header
        int headerSize = dis.readInt();
        byte[] headerBytes = new byte[headerSize];
        dis.read(headerBytes);
        FileHeader header = FileHeader.deserialize(headerBytes);

        // Read compressed data
        byte[] compressedData = dis.readAllBytes();

        // Find correct pipeline
        CompressionPipeline pipeline =
            getPipeline(header.getPipelineName());

        // Decompress
        byte[] restoredData = pipeline.decompress(compressedData);

        // Verify checksum
        CRC32 crc = new CRC32();
        crc.update(restoredData);
        int checksum = (int) crc.getValue();

        if (checksum != header.getChecksum()) {
            throw new IOException("File corrupted! Checksum mismatch.");
        }

        // Save restored file
        String outputPath = outputFolder +
            "/restored" + header.getOriginalExtension();
        fileHandler.writeFile(outputPath, restoredData);

        System.out.println("Decompressed successfully: " + outputPath);
        System.out.println("Integrity: PASSED ✅");
    }

```

```

// Benchmark all 6 pipelines on same file
public List<CompressionResult> benchmarkAll(
    String filePath) throws IOException {

    byte[] originalData = fileHandler.readFile(filePath);
    List<CompressionResult> results = new ArrayList<>();

    for (CompressionPipeline pipeline : allPipelines) {

        long startCompress = System.currentTimeMillis();
        byte[] compressed = pipeline.compress(originalData);
        long compressTime = System.currentTimeMillis() - startCompress;

        long startDecompress = System.currentTimeMillis();
        byte[] decompressed = pipeline.decompress(compressed);
        long decompressTime =
            System.currentTimeMillis() - startDecompress;

        boolean integrity =
            pipeline.verifyIntegrity(originalData, decompressed);

        results.add(new CompressionResult(
            pipeline.getPipelineName(),
            originalData.length,
            compressed.length,
            compressTime,
            decompressTime,
            integrity
        ));
    }

    return results;
}

private CompressionPipeline getPipeline(String name) {
    return allPipelines.stream()
        .filter(p -> p.getPipelineName().equals(name))
        .findFirst()
        .orElse(allPipelines.get(2)); // default Huffman
}

public List<CompressionPipeline> getAllPipelines() {
    return allPipelines;
}

```

```
}  
}
```

WEEK 3 — GUI

DAY 15, 16 & 17 — JavaFX GUI

Main Window: `gui/MainWindow.java`

```
java  
  
package com.compressor.gui;  
  
import com.compressor.core.*;  
import javafx.application.Application;  
import javafx.geometry.Insets;  
import javafx.scene.Scene;  
import javafx.scene.control.*;  
import javafx.scene.layout.*;  
import javafx.stage.FileChooser;  
import javafx.stage.Stage;  
import java.io.File;  
  
public class MainWindow extends Application {  
  
    private MultiLevelCompressor compressor;  
    private Label fileLabel;  
    private Label fileSizeLabel;  
    private String selectedFilePath;  
    private ToggleGroup pipelineGroup;  
  
    @Override  
    public void start(Stage stage) {  
  
        compressor = new MultiLevelCompressor();  
  
        // Title  
        Label title = new Label("Multi-Level File Compressor");  
        title.setStyle("-fx-font-size: 20px; -fx-font-weight: bold;");  
  
        // File drop area  
        VBox dropArea = new VBox();
```

```

dropArea.setPrefHeight(120);
dropArea.setStyle(
    "-fx-border-color: gray; -fx-border-style: dashed;" +
    "-fx-border-width: 2; -fx-alignment: center;"
);
Label dropLabel = new Label("Drag & Drop File Here");
Button browseBtn = new Button("Browse File");
dropArea.getChildren().addAll(dropLabel, browseBtn);

// File info
fileLabel = new Label("No file selected");
fileSizeLabel = new Label("");

// Pipeline selection
Label pipelineLabel = new Label("Select Pipeline:");
pipelineGroup = new ToggleGroup();
String[] pipelines = {
    "RLE", "LZW", "Huffman",
    "RLE→Huffman", "LZW→Huffman", "RLE→LZW→Huffman"
};

VBox pipelineBox = new VBox(5);
for (String p : pipelines) {
    RadioButton rb = new RadioButton(p);
    rb.setToggleGroup(pipelineGroup);
    if (p.equals("RLE→LZW→Huffman")) rb.setSelected(true);
    pipelineBox.getChildren().add(rb);
}

// Buttons
Button compressBtn = new Button("COMPRESS");
Button decompressBtn = new Button("DECOMPRESS");
Button compareBtn = new Button("COMPARE ALL PIPELINES");

compressBtn.setStyle(
    "-fx-background-color: #2196F3; -fx-text-fill: white;" +
    "-fx-font-size: 14px; -fx-pref-width: 150;"
);
decompressBtn.setStyle(
    "-fx-background-color: #4CAF50; -fx-text-fill: white;" +
    "-fx-font-size: 14px; -fx-pref-width: 150;"
);
compareBtn.setStyle(
    "-fx-background-color: #FF9800; -fx-text-fill: white;" +
    "-fx-font-size: 14px;"
);

```

```

HBox buttonBox = new HBox(10, compressBtn, decompressBtn);

// Status label
Label statusLabel = new Label("");

// Layout
VBox root = new VBox(15);
root.setPadding(new Insets(20));
root.getChildren().addAll(
    title, dropArea, fileLabel, fileSizeLabel,
    pipelineLabel, pipelineBox,
    buttonBox, compareBtn, statusLabel
);

// Browse button action
browseBtn.setOnAction(e -> {
    FileChooser chooser = new FileChooser();
    File file = chooser.showOpenDialog(stage);
    if (file != null) {
        selectedFilePath = file.getAbsolutePath();
        fileLabel.setText("Selected: " + file.getName());
        fileSizeLabel.setText(
            "Size: " + file.length() / 1024 + " KB"
        );
    }
});

// Drag and drop
dropArea.setOnDragOver(e -> {
    if (e.getDragboard().hasFiles()) {
        e.acceptTransferModes(
            javafx.scene.input.TransferMode.COPY
        );
    }
    e.consume();
});

dropArea.setOnDragDropped(e -> {
    if (e.getDragboard().hasFiles()) {
        File file = e.getDragboard().getFiles().get(0);
        selectedFilePath = file.getAbsolutePath();
        fileLabel.setText("Selected: " + file.getName());
        fileSizeLabel.setText(
            "Size: " + file.length() / 1024 + " KB"
        );
    }
});

```

```

    }
    e.consume();
});

// Compress button action
compressBtn.setOnAction(e -> {
    if (selectedFilePath == null) {
        statusLabel.setText("Please select a file first!");
        return;
    }
    RadioButton selected =
        (RadioButton) pipelineGroup.getSelectedToggle();
    String pipeline = selected.getText();

    try {
        String output = selectedFilePath
            .substring(0, selectedFilePath.lastIndexOf('.'))
            + ".mlc";
        compressor.compressFile(
            selectedFilePath, output, pipeline
        );
        statusLabel.setText("✅ Compressed: " + output);
    } catch (Exception ex) {
        statusLabel.setText("❌ Error: " + ex.getMessage());
    }
});

// Decompress button action
decompressBtn.setOnAction(e -> {
    FileChooser chooser = new FileChooser();
    chooser.getExtensionFilters().add(
        new FileChooser.ExtensionFilter("MLC Files", "*.mlc")
    );
    File file = chooser.showOpenDialog(stage);
    if (file != null) {
        try {
            String folder = file.getParent();
            compressor.decompressFile(
                file.getAbsolutePath(), folder
            );
            statusLabel.setText("✅ Decompressed successfully!");
        } catch (Exception ex) {
            statusLabel.setText("❌ Error: " + ex.getMessage());
        }
    }
});

```

```

// Compare all button
compareBtn.setOnAction(e -> {
    if (selectedFilePath == null) {
        statusLabel.setText("Please select a file first!");
        return;
    }
    try {
        var results = compressor.benchmarkAll(selectedFilePath);
        new ResultsDashboard(results).show();
    } catch (Exception ex) {
        statusLabel.setText("✖ Error: " + ex.getMessage());
    }
});

Scene scene = new Scene(root, 450, 650);
stage.setTitle("Multi-Level File Compressor");
stage.setScene(scene);
stage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

Results Dashboard: `gui/ResultsDashboard.java`

```

java

package com.compressor.gui;

import com.compressor.core.CompressionResult;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;
import java.util.List;

public class ResultsDashboard {

    private List<CompressionResult> results;

    public ResultsDashboard(List<CompressionResult> results) {
        this.results = results;
    }
}

```



```
public void show() {

    Stage stage = new Stage();
    stage.setTitle("Pipeline Comparison");

    // Table
    TableView<CompressionResult> table = new TableView<>();

    TableColumn<CompressionResult, String> nameCol =
        new TableColumn<>("Pipeline");
    nameCol.setCellValueFactory(d ->
        new javafx.beans.property.SimpleStringProperty(
            d.getValue().getPipelineName()
        )
    );

    TableColumn<CompressionResult, String> sizeCol =
        new TableColumn<>("Compressed Size");
    sizeCol.setCellValueFactory(d ->
        new javafx.beans.property.SimpleStringProperty(
            d.getValue().getCompressedSize() / 1024 + " KB"
        )
    );

    TableColumn<CompressionResult, String> ratioCol =
        new TableColumn<>("Ratio");
    ratioCol.setCellValueFactory(d ->
        new javafx.beans.property.SimpleStringProperty(
            String.format("%.1f%%",
                d.getValue().getCompressionRatio())
        )
    );

    TableColumn<CompressionResult, String> timeCol =
        new TableColumn<>("Time");
    timeCol.setCellValueFactory(d ->
        new javafx.beans.property.SimpleStringProperty(
            d.getValue().getCompressionTimeMs() + " ms"
        )
    );

    TableColumn<CompressionResult, String> integrityCol =
        new TableColumn<>("Integrity");
    integrityCol.setCellValueFactory(d ->
        new javafx.beans.property.SimpleStringProperty(
```

```

        d.getValue().isIntegrityPassed() ? "✅ PASS" : "❌ FAIL"
    )
);

table.getColumns().addAll(
    nameCol, sizeCol, ratioCol, timeCol, integrityCol
);
table.getItems().addAll(results);

VBox root = new VBox(10, new Label("Pipeline Comparison"), table);
root.setPadding(new javafx.geometry.Insets(15));

stage.setScene(new Scene(root, 600, 300));
stage.show();
}
}

```

DAY 18, 19 & 20 — Testing Everything

Test every single scenario:

- ✅ Compress a .txt file → decompress → verify match
- ✅ Compress a .png file → decompress → verify match
- ✅ Compress a .pdf file → decompress → verify match
- ✅ All 6 pipelines compress and decompress correctly
- ✅ Benchmark runs without crashing
- ✅ Drag and drop works
- ✅ Browse button works
- ✅ Wrong file shows error
- ✅ Results dashboard shows correct numbers
- ✅ .mlc file is actually smaller than original

DAY 21 — Final Polish

- ✅ Add application icon
- ✅ Clean up all System.out.println
- ✅ Handle all exceptions with proper error messages
- ✅ Test on 5 different file types
- ✅ Take screenshots for documentation
- ✅ Write simple README

- ✓ Draw UML class diagram
- ✓ Prepare demo script for sir

Summary — What You Build Day by Day

Day 1 → Abstract base class
Day 2 → RLE compress + decompress + test
Day 3-4 → LZW compress + decompress + test
Day 5-7 → Huffman compress + decompress + test
Day 8-9 → Pipeline + CompressionResult
Day 10-11 → File handler + custom .mlc format
Day 12-13 → Main compressor engine
Day 14 → Connect everything + integration test
Day 15-17 → JavaFX GUI
Day 18-19 → Results dashboard + benchmark view
Day 20 → Full testing all file types
Day 21 → Polish + documentation + demo prep

Follow this exactly in this order. Do not skip ahead. Do not touch GUI before Day 15. Build and test each piece before moving to the next.

My group has two members can you perfectly divide it for two people this project. No one gets difficult work both will do same difficulty work. Just divide in best possible way and tell me how to work together .

Perfect Division for Two Members

The Core Principle of Division

Person A → Backend (Algorithms + Core Engine)
Person B → Frontend + Integration (File System + GUI)

Both are equal difficulty.
Both require real understanding.
Both cannot be done without thinking.
Neither can finish without the other.

Person A — Algorithm Developer

Your Complete Responsibility

Week 1:

- Day 1 → Abstract base class (CompressionAlgorithm.java)
- Day 2 → RLE compress + decompress + test
- Day 3 → LZW compress + decompress + test
- Day 4 → LZW decompress edge cases + full test
- Day 5 → Huffman Node + Priority Queue + Tree builder
- Day 6 → Huffman encoder + bit packing
- Day 7 → Huffman decoder + tree serializer + full test

Week 2:

- Day 8 → CompressionPipeline.java (chain algorithms)
- Day 9 → All 6 pipeline combinations built and tested
- Day 10 → CompressionResult.java (stats model)
- Day 11 → BenchmarkEngine (run all 6 pipelines, measure time)

Week 3:

- Day 15 → Help Person B connect algorithms to GUI
- Day 16 → Fix any algorithm bugs found during GUI testing
- Day 17 → Test all algorithms on different file types
- Day 18 → Edge case testing (empty file, huge file, 1 byte)
- Day 19 → Write algorithm explanation documentation
- Day 20 → Prepare viva answers for algorithm questions
- Day 21 → Final testing + demo support

Files You Own

```
com.compressor.algorithms/  
├─ CompressionAlgorithm.java    ← you write  
├─ RLECompressor.java          ← you write  
├─ LZWCompressor.java          ← you write  
└─ HuffmanCompressor.java      ← you write  
  
com.compressor.core/  
├─ CompressionPipeline.java    ← you write  
├─ CompressionResult.java      ← you write  
└─ BenchmarkEngine.java        ← you write
```

What Makes Your Part Hard

- Huffman bit packing logic is genuinely difficult
- LZW dictionary reconstruction requires careful thinking
- Pipeline reverse decompression order must be perfect
- One bug in any algorithm breaks the entire project
- You must test every algorithm thoroughly before Week 3

Person B — System & GUI Developer

Your Complete Responsibility

Week 1:

- Day 1 → Set up IntelliJ project structure for both members
- Day 2 → FileHandler.java (read/write any file as bytes)
- Day 3 → FileHeader.java (custom .mlc format design)
- Day 4 → FileHeader serialize + deserialize + test
- Day 5 → CRC32 checksum calculation + verification
- Day 6 → MultilevelCompressor.java (main engine)
- Day 7 → Connect FileHeader + FileHandler + test saving .mlc

Week 2:

- Day 8 → JavaFX main window layout (no logic yet, just UI)
- Day 9 → Drag and drop + browse button functionality
- Day 10 → Connect compress button to Person A's pipeline
- Day 11 → Connect decompress button + file restoration

Week 3:

- Day 15 → ResultsDashboard (benchmark comparison table)
- Day 16 → Session history log panel
- Day 17 → Progress indicator during compression
- Day 18 → Auto open restored file after decompression
- Day 19 → Error handling + user friendly messages
- Day 20 → Full GUI testing + screenshots for documentation
- Day 21 → Final polish + demo preparation

Files You Own

```
com.compressor.io/  
└─ FileHandler.java           ← you write  
  
com.compressor.core/  
├─ FileHeader.java           ← you write  
└─ MultilevelCompressor.java ← you write
```

```
com.compressor.gui/  
├─ MainWindow.java          ← you write  
├─ ResultsDashboard.java   ← you write  
└─ ProgressPanel.java      ← you write
```

What Makes Your Part Hard

- Custom .mlc file format design requires systems thinking
- Header serialize/deserialize must be perfectly symmetric
- CRC32 checksum must match exactly or integrity fails
- JavaFX drag and drop has tricky event handling
- Connecting Person A's algorithms correctly is critical
- If file reading/writing has bugs nothing works at all

How To Work Together — Step by Step

Step 1 — Day 1 Together (Both Members)

Sit together on Day 1.
Person B sets up the IntelliJ project.
Both agree on exact package names.
Both agree on exact class names.
Both agree on method signatures.

This is the most important meeting of the project.
Everything built later depends on what you agree today.

Agree on this interface contract together:

```
java  
  
// Person A will build this  
public abstract class CompressionAlgorithm {  
    public abstract byte[] compress(byte[] data);  
    public abstract byte[] decompress(byte[] data);  
    public abstract String getName();  
}  
  
// Person A will build this  
public class CompressionPipeline {
```

```
public byte[] compress(byte[] data) { ... }  
public byte[] decompress(byte[] data) { ... }  
}  
  
// Person B will call these methods  
// Person B does NOT change these method signatures  
// Person A does NOT change these method signatures  
// Both agreed → contract is locked
```

Step 2 — Work Separately Week 1 and Week 2

Person A works on algorithms independently.
Person B works on file system independently.

You do NOT need each other during Week 1.
Person B can test FileHandler with dummy byte arrays.
Person A can test algorithms with hardcoded strings.

Meet every 2 days for 15 minutes:

- What did you finish?
- Any problems?
- Anything changed in the contract?

Step 3 — Integration Day (Day 14 Together)

This is the second critical meeting.
Both sit together for a full day.

Person A brings: all algorithms + pipeline working + tested
Person B brings: file system + GUI skeleton working + tested

Together you:

1. Connect Person B's file reading to Person A's pipeline
2. Connect Person A's CompressionResult to Person B's GUI table
3. Run full test: load real file → compress → save .mlc → decompress → verify
4. Fix anything that breaks together

This day is non negotiable. Both must be present.

Step 4 — Week 3 Together

Week 3 you work closely together every day.
GUI needs algorithm output to display results.
Algorithms need GUI testing to find edge case bugs.

- Daily 30 minute meeting:
- What broke yesterday?
 - What needs to be finished today?
 - Who is blocking who?

Step 5 — Final Week Division

- Person A prepares:
- Algorithm explanation (how RLE LZW Huffman work)
 - UML class diagram for algorithms side
 - Viva answers for algorithm questions
 - Test results document

- Person B prepares:
- GUI screenshots
 - UML class diagram for system side
 - Demo script (who clicks what during presentation)
 - Project README

Viva Division — Who Answers What

Sir's Question	Who Answers
How does Huffman tree work?	Person A
Why does LZW not save dictionary?	Person A
How do you chain algorithms?	Person A
What is stored in .mlc file?	Person B
How does drag and drop work?	Person B
How do you verify file integrity?	Person B
What is the Strategy design pattern?	Both

Sir's Question

Who Answers

What OOP concepts did you use?

Both

Why is your project better than management systems?

Both

Golden Rules for Working Together

Rule 1 – Never change agreed method signatures without telling your partner first.

Rule 2 – Test YOUR part before integration day.
Do not bring broken code to integration.

Rule 3 – Use WhatsApp or any chat to share code files daily via GitHub or Google Drive.

Rule 4 – If you are stuck tell your partner same day. Do not stay stuck silently for 3 days.

Rule 5 – Both must understand the WHOLE project not just your own part.
Sir will ask anyone anything during viva.

Difficulty Comparison — Is It Really Equal?

Person A – Algorithm Side:

RLE → Medium
LZW → Medium-Hard
Huffman → Hard
Pipeline → Medium
Overall → Hard 

Person B – System Side:

FileHandler → Medium
FileHeader → Medium-Hard
.mlc format → Hard
JavaFX GUI → Medium-Hard
Integration → Hard
Overall → Hard 

Both equally challenging. Both require genuine thinking. Neither can be faked.

Recommended Tool for Sharing Code

Use GitHub (best option):

1. Person B creates repository on github.com
2. Shares link with Person A
3. Both push their code daily
4. Never overwrite each other's files
5. Full history of who did what – useful for viva proof

If GitHub is too complex use Google Drive:

Share one folder

Each person keeps their own package folder

Upload updated files daily

Never edit same file at same time

Summary Card — Stick This On Your Wall

PERSON A

RLECompressor
LZWCompressor
HuffmanCompressor
CompressionPipeline
CompressionResult
BenchmarkEngine

PERSON B

FileHandler
FileHeader (.mlc format)
MultiLevelCompressor
MainWindow (JavaFX)
ResultsDashboard
ProgressPanel

TOGETHER ON:

Day 1 → Project setup + contract agreement
Day 14 → Integration day
Week 3 → Daily collaboration
Day 21 → Demo preparation

This division is fair, clean, and professional. Both members contribute equally. Both members must understand the full project. Both members will have genuine work to show and explain during viva.

You divide in a way that one will do integration and ui and other will do Algorithm make it in a way both will do both both will work on algorithm and both will work on ui and integratio

Redesigned Division — Both Do Everything

The New Principle

Each person builds ONE complete algorithm from scratch including its own file handling and its own part of the GUI.

Both touch algorithms. Both touch GUI. Both touch integration. Neither person is just "the UI guy" or "the algorithm guy".

How to Split — By Feature Ownership

Person A owns → RLE + LZW + their part of GUI

Person B owns → Huffman + Core System + their part of GUI

Shared together → Abstract base class, Pipeline, Final integration

Person A — Full Breakdown

Algorithms

RLECompressor.java

- compress() method
- decompress() method
- test RLE on text files
- test RLE on binary files

LZWCompressor.java

- compress() with dictionary building
- decompress() with dictionary reconstruction

- test LZW on text files
- test LZW on binary files

File System (Person A owns this part)

FileHandler.java

- readFile(path) – read any file as bytes
- writeFile(path, bytes) – write bytes to any file
- getExtension(path) – get .pdf .txt .png
- getFileName(path) – get name without extension
- getFileSizeKB(path) – get size in KB

GUI (Person A owns these screens)

MainWindow.java – the first screen user sees

- Application title and layout
- Drag and drop area
- Browse file button
- File name and size display
- Pipeline selection radio buttons
- Compress and Decompress buttons
- Status message label

ProgressPanel.java – shown during compression

- Progress bar
- Current stage label (RLE running... LZW running...)
- Time elapsed counter

Integration Person A Does

- Connect Browse button to FileHandler.readFile()
- Connect Drag and Drop to file loading
- Connect Compress button to pipeline.compress()
- Show compression status after completion
- Connect Decompress button to pipeline.decompress()

Person B — Full Breakdown

Algorithm

HuffmanCompressor.java (the hardest one)

- HuffmanNode inner class
- Frequency counter
- Priority queue and tree builder
- Code table generator
- Bit packer (compress)
- Tree serializer (save tree to bytes)
- Bit reader and tree traversal (decompress)
- Tree deserializer (rebuild tree from bytes)
- Full test compress + decompress

Core System (Person B owns this)

FileHeader.java

- Design the .mlc file structure
- Store: magic number, pipeline name,
original extension, original size,
CRC32 checksum, timestamp
- serialize() – convert header to bytes
- deserialize() – read bytes back to header

MultiLevelCompressor.java

- compressFile() – read file, compress, build
header, save .mlc
- decompressFile() – read .mlc, verify header,
decompress, verify checksum,
save restored file
- CRC32 checksum calculation and verification

GUI (Person B owns these screens)

ResultsDashboard.java – benchmark comparison screen

- Table with all 6 pipeline results
- Columns: Pipeline, Size, Ratio, Time, Integrity
- Best pipeline recommendation label
- Visual bar chart of compression ratios

HistoryPanel.java – session log screen

- List of all files compressed this session
- Shows file name, algorithm, ratio, time, status
- Clear history button

Integration Person B Does

- Connect Compare All button to benchmarkAll()
- Connect results to ResultsDashboard table
- Auto open restored file after decompression
- Connect CRC32 checksum to integrity display
- Connect session history log to HistoryPanel

Shared Work — Both Do Together

Day 1 Together — Foundation

Both sit together and build:

CompressionAlgorithm.java (abstract base)

- Both agree on compress() signature
- Both agree on decompress() signature
- Both write it together line by line
- Both understand it completely

Agree on all package names and class names.

This takes 1-2 hours maximum.

Lock the contract. Do not change later.

Day 9 Together — Pipeline

Both sit together and build:

CompressionPipeline.java

- addStage() – Person A explains their algorithms
- compress() – chain all stages in order
- decompress() – reverse order, Person B explains Huffman
- verifyIntegrity() – Arrays.equals check
- Build all 6 pipeline combinations together
- Test pipeline end to end together

BenchmarkEngine.java

- Run all 6 pipelines on same file
- Measure time for each
- Return list of CompressionResult
- Both write this together

Day 14 Together — Full Integration

Both sit together for full day:

Person A brings:

- ✓ RLE working and tested
- ✓ LZW working and tested
- ✓ FileHandler working and tested
- ✓ MainWindow UI built

Person B brings:

- ✓ Huffman working and tested
- ✓ FileHeader working and tested
- ✓ MultiLevelCompressor working
- ✓ ResultsDashboard built

Together:

- Connect MainWindow compress button to MultiLevelCompressor.compressFile()
- Connect compare button to BenchmarkEngine + ResultsDashboard
- Run real file through entire system
- Fix whatever breaks
- Verify .mlc file saves correctly
- Verify decompression restores perfectly

Day 21 Together — Final Demo Prep

Both prepare demo together:

- Agree on who clicks what
- Agree on who speaks what
- Practice full demo 3 times
- Prepare viva answers for each other's parts

Week by Week Schedule

Week 1

Day 1 – BOTH TOGETHER

- Build abstract base class
- Set up project structure
- Agree on all contracts

Day 2

Person A → Start RLE compress()

Person B → Start HuffmanNode + frequency counter

Day 3

Person A → RLE decompress() + test RLE

Person B → Huffman priority queue + tree builder

Day 4

Person A → Start LZW compress() dictionary building

Person B → Huffman code generator + bit packer

Day 5

Person A → LZW decompress() dictionary reconstruction

Person B → Huffman decompress() + tree serializer

Day 6

Person A → Test LZW fully + fix bugs

Start FileHandler.java

Person B → Test Huffman fully + fix bugs

Start FileHeader.java

Day 7

Person A → Finish FileHandler + test file reading

Person B → Finish FileHeader serialize/deserialize

Evening – Quick 30 min meeting:

Share code, confirm algorithms work,
confirm file classes work

Week 2

Day 8

Person A → Start MainWindow.java UI layout

Person B → Start MultiLevelCompressor.java

Day 9 – BOTH TOGETHER

Build CompressionPipeline.java together

Build all 6 combinations together

Build BenchmarkEngine together

Test pipeline with all three algorithms

Day 10

Person A → Add drag and drop to MainWindow

Add browse button functionality

Person B → Finish compressFile() in MultiLevelCompressor

Add CRC32 checksum logic

Day 11

Person A → Add pipeline radio buttons to MainWindow

Connect compress button (basic connection)

Person B → Finish decompressFile() in MultiLevelCompressor

Add checksum verification

Day 12

Person A → Build ProgressPanel.java

Person B → Start ResultsDashboard.java table

Day 13

Person A → Connect ProgressPanel to MainWindow

Person B → Finish ResultsDashboard + bar chart

Day 14 – BOTH TOGETHER (Integration Day)

Full system connection

Real file testing

Fix all bugs together

Week 3

Day 15

Person A → Fix any MainWindow bugs from integration

Add status messages and error handling

Person B → Build HistoryPanel.java session log

Connect history to compression events

Day 16

Person A → Test RLE + LZW on image files and PDFs

Fix any bugs found

Person B → Test Huffman on image files and PDFs

Fix any bugs found

Day 17

Person A → Add auto open restored file feature

Person B → Add best pipeline recommendation label
in ResultsDashboard

Day 18

Person A → Edge case testing
(empty file, 1 byte file, very large file)

Person B → Edge case testing
(corrupt .mlc file, wrong file type)

Day 19

Person A → Write documentation for RLE and LZW
Draw UML for your classes

Person B → Write documentation for Huffman and core
Draw UML for your classes

Day 20

Both → Combine UML diagrams
Take screenshots of full working app
Write README together

Day 21 – BOTH TOGETHER

Practice demo 3 times
Prepare viva answers
Final submission

Viva — Who Answers What

Sir's Question	Who Answers	
How does RLE work?	Try Claude yourself	Person A